

activities comply with regulations by alerting firms to any violations, allowing them to avoid penalties and reputational damage.

Additionally, data governance is essential for preserving compliance, privacy, and data quality. To accelerate data workflows, guarantee data consistency, and shorten deployment times, frameworks such as DataOps combine data engineering with agile principles (Lawton, 2022). Organizations may improve data access, security, and integrity across teams and platforms by using standardized data lifecycle management procedures.

This can be necessary because according to the survey findings in figure 7, we saw that a 17% of the participants claims that data governance and compliance can be a significant challenge for organizations as it can lead to severe damages to the organization.

IMPLEMENTATION

Sales dashboard web app for Adidas

The sales dashboard web app represents interactive data visualizations, such as line, bar, pie, and heatmap charts and graphs, to show product performance, regional distributions, and sales trends of Adidas.

The web app offers a complete solution for data-driven decision-making by integrating Prophet for predictive analytics, Flask for authentication, and Streamlit for visualization.

This chapter documents the technical implementation of the dashboard, which integrates realtime data visualization, user authentication, and predictive forecasting into a single application.

Dataset Overview

The Adidas US Sales Dataset from Kaggle, which includes comprehensive sales statistics from 2020–2021, is used in the analysis and for the development of the web app. The dataset consists of:

- 9,649 transactions across US retailers
- Key columns: Retailer, RetailerID, InvoiceDate, Region, State, City, Product, PriceperUnit, UnitsSold, TotalSales, OperatingProfit, OperatingMargin, SalesMethod

Retailer	RetailerID	InvoiceDate	Region	State	City	Product	PriceperUnit	UnitsSold	TotalSales	OperatingProfit	OperatingMargin	SalesMethod
Foot Locker	1185732	2/3/2020	Northeast	New York	New York	Women's Athletic	\$45.00	825	\$37,125	\$12,994	35%	Outlet
Foot Locker	1185732	2/4/2020	Northeast	New York	New York	Men's Apparel	\$60.00	825	\$49,500	\$14,850	30%	Outlet
Foot Locker	1185732	2/5/2020	Northeast	New York	New York	Women's Apparel	\$50.00	950	\$47,500	\$11,875	25%	Outlet
Foot Locker	1185732	2/6/2020	Northeast	New York	New York	Men's Street Foot	\$60.00	1,220	\$73,200	\$36,600	50%	Outlet
Foot Locker	1185732	2/7/2020	Northeast	New York	New York	Men's Athletic FC	\$55.00	925	\$50,875	\$15,263	30%	Outlet
Foot Locker	1185732	2/8/2020	Northeast	New York	New York	Women's Street	\$50.00	900	\$45,000	\$15,750	35%	Outlet
Foot Locker	1185732	1/1/2020	Northeast	New York	New York	Men's Street Foot	\$50.00	1,200	\$60,000	\$30,000	50%	In-store
Foot Locker	1185732	1/2/2020	Northeast	New York	New York	Men's Athletic FC	\$50.00	1,000	\$50,000	\$15,000	30%	In-store
Foot Locker	1185732	1/3/2020	Northeast	New York	New York	Women's Street	\$40.00	1,000	\$40,000	\$14,000	35%	In-store
Foot Locker	1185732	1/4/2020	Northeast	New York	New York	Women's Athletic	\$45.00	850	\$38,250	\$13,388	35%	In-store
Foot Locker	1185732	1/5/2020	Northeast	New York	New York	Men's Apparel	\$60.00	900	\$54,000	\$16,200	30%	In-store
Foot Locker	1185732	1/6/2020	Northeast	New York	New York	Women's Apparel	\$50.00	1,000	\$50,000	\$12,500	25%	In-store
Foot Locker	1185732	1/7/2020	Northeast	New York	New York	Men's Street Foot	\$50.00	1,250	\$62,500	\$31,250	50%	In-store
Foot Locker	1185732	1/8/2020	Northeast	New York	New York	Men's Athletic FC	\$50.00	900	\$45,000	\$13,500	30%	Outlet
Foot Locker	1185732	1/11/2020	Northeast	New York	New York	Women's Street	\$40.00	950	\$38,000	\$13,300	35%	Outlet
Foot Locker	1185732	1/22/2020	Northeast	New York	New York	Women's Athletic	\$45.00	825	\$37,125	\$12,994	35%	Outlet
Foot Locker	1185732	1/23/2020	Northeast	New York	New York	Men's Apparel	\$60.00	900	\$54,000	\$16,200	30%	Outlet
Foot Locker	1185732	1/24/2020	Northeast	New York	New York	Women's Apparel	\$50.00	1,000	\$50,000	\$12,500	25%	Outlet
Foot Locker	1185732	1/25/2020	Northeast	New York	New York	Men's Street Foot	\$50.00	1,220	\$61,000	\$30,500	50%	Outlet
Foot Locker	1185732	1/26/2020	Northeast	New York	New York	Men's Athletic FC	\$50.00	925	\$46,250	\$13,875	30%	Outlet
Foot Locker	1185732	1/27/2020	Northeast	New York	New York	Women's Street	\$40.00	950	\$38,000	\$13,300	35%	Outlet
Foot Locker	1185732	1/28/2020	Northeast	New York	New York	Women's Athletic	\$45.00	800	\$36,000	\$12,600	35%	Outlet
Foot Locker	1185732	1/29/2020	Northeast	New York	New York	Men's Apparel	\$60.00	850	\$51,000	\$15,300	30%	Outlet
Foot Locker	1185732	1/30/2020	Northeast	New York	New York	Women's Apparel	\$50.00	950	\$47,500	\$11,875	25%	Outlet
Foot Locker	1185732	1/31/2020	Northeast	New York	New York	Men's Street Foot	\$50.00	1,200	\$60,000	\$30,000	50%	Outlet
Foot Locker	1185732	2/1/2020	Northeast	New York	New York	Men's Athletic FC	\$50.00	900	\$45,000	\$13,500	30%	Outlet
Foot Locker	1185732	2/2/2020	Northeast	New York	New York	Women's Street	\$40.00	900	\$36,000	\$12,600	35%	Outlet
Foot Locker	1185732	2/9/2020	Northeast	New York	New York	Women's Athletic	\$50.00	850	\$42,500	\$14,875	35%	Outlet
Foot Locker	1185732	2/10/2020	Northeast	New York	New York	Men's Apparel	\$60.00	875	\$52,500	\$15,750	30%	Outlet

Figure 27

System Architecture

The application follows a modular three-tier architecture:

- Frontend:
 - Streamlit for interactive dashboards (primary interface).
 - HTML/CSS for Flask-based authentication pages.
- Backend:
 - Flask handles user authentication and session management.
 - Streamlit processes data and renders visualizations.
- Data Layer:
 - Pandas for data manipulation.
 - MongoDB (via pymongo) for user credential storage.
 - Facebook Prophet for time-series forecasting.

Component	Technology
Frontend Framework	Streamlit, Plotly, HTML
Backend Framework	Flask
Database	MongoDB
Predictive analysis	Prophet, Random Forest Regressor

Figure 28

Implementation steps

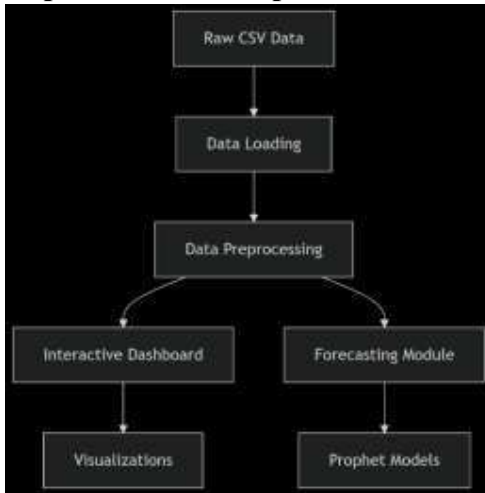


Figure 29

- Data ingestion and preprocessing

Dataset: Adidas sales dataset

- DateTime conversion for time-series analysis.
- Dynamic filtering by year, month, product, retailer, and region (user-selectable).

The excel file is read and imported using pandas.

```

# reading the data from excel file
df = pd.read_excel("Adidas.xlsx")
st.set_page_config(layout="wide")
st.markdown('<style>div.block-container{padding-top:1rem;}</style>', unsafe_allow_html=True)

```

Figure 31

```
# Apply filters to the dataframe
filtered_df = df[
    (df['InvoiceDate'].dt.year.isin(selected_years)) &
    (df['InvoiceDate'].dt.strftime('%B').isin(selected_months)) &
    (df['Product'].isin(selected_products)) &
    (df['Retailer'].isin(selected_retailers)) &
    (df['Region'].isin(selected_regions))
]
```

Figure 30

- Authentication system (Flask)

User management

- MongoDB stores hashed passwords (werkzeug.security).
- Flask-Login handles session management.

Key Routes:

- /login and /register (HTML templates).
- /dashboard launches the Streamlit app in a separate thread

```
def run_streamlit():
    # Get the path to your Streamlit app
    streamlit_path = os.path.join(os.path.dirname(os.path.dirname(__file__)), 'test6.py')
    subprocess.run(['streamlit', 'run', streamlit_path])

@app.route('/')
def index():
    if current_user.is_authenticated:
        return redirect(url_for('dashboard'))
    return redirect(url_for('login'))

@app.route('/login', methods=['GET', 'POST'])
def login():
    if current_user.is_authenticated:
        return redirect(url_for('dashboard'))

    if request.method == 'POST':
        username = request.form.get('username')
        password = request.form.get('password')

        user_data = users.find_one({'username': username})
        print("User data from DB:", user_data)
        if user_data:
            print("Password in DB:", user_data['password'])
            print("Password entered:", password)
            print("Password check:", check_password_hash(user_data['password'], password))
            if user_data and check_password_hash(user_data['password'], password):
                user = User(user_data)
                login_user(user)
                return redirect(url_for('dashboard'))
```

Figure 32

- User interface (Interactive dashboard) The dashboard features a three-page layout with:

Dashboard Page:

- Interactive filters (year, month, product, retailer, region)
- KPI metrics (total sales, units sold, profit)
- Multiple visualization types (bar charts, line charts, treemaps)
- Data export functionality

Predictive Analysis Page:

- Time series forecasting using Facebook Prophet
- Model evaluation metrics (MAE, RMSE, MAPE)
- Visualization of forecast vs actuals

Advance Predictive Analysis Page:

- Time series forecasting using Random Forest Regressor
 - Model evaluation metrics (MAE, RMSE, MAPE)
 - Visualization of forecast vs actuals
- Predictive analytics

Forecasting Workflow:

- Aggregate sales data by date.
- Train Prophet model on historical data.
- Generate forecasts (yhat, confidence intervals).

```
elif page == "Predictive Analysis":
    st.title("Predictive Analysis: Future Sales Forecast")
    st.markdown("---")
    st.info("This section uses Facebook Prophet to forecast future sales based on historical data.")

    # Prepare data for Prophet
    sales_df = df.copy()
    sales_df = sales_df.groupby('InvoiceDate')['TotalSales'].sum().reset_index()
    sales_df = sales_df.rename(columns={"InvoiceDate": "ds", "TotalSales": "y"})
    sales_df = sales_df.sort_values("ds")

    # Prophet expects datetime
    sales_df["ds"] = pd.to_datetime(sales_df["ds"])

    # Sidebar: Forecast horizon
    periods = st.sidebar.selectbox("Forecast Horizon (months)", [3, 6, 12], index=1)

    # Train Prophet model
    m = Prophet()
    m.fit(sales_df)

    # Make future dataframe
    future = m.make_future_dataframe(periods=periods, freq='M')
    forecast = m.predict(future)
```

Figure 33

Pages and user flow

Login page

- Location: /login (Flask route)

- Purpose: Secure authentication for dashboard access.
- Form structure:
 - Submission of the POST method form to the /login endpoint
 - Password and username are input fields
 - Both fields must have this property for client-side validation.
 - Click the "Submit" button to begin the login procedure.
- Key features:
- Username/Password Input:
 - Form validation ensures fields are filled.
 - MongoDB verifies credentials using `check_password_hash()`.

- Error Handling:

Displays flash messages (e.g., "Invalid username/password").

- Navigation:
 - Register Link: Redirects to /register for new users.
 - Login Button: Authenticates and redirects to /dashboard.
- Backend processing flow:

The following procedure takes place after the form is submitted:

1. Using the username and password, the browser submits a POST request to the server's /login endpoint.
2. The credentials are sent to the Flask-implemented server-side route handler.
3. Validation is done by the backend:
 - Verifies whether the username is present in the database.
 - Confirms that the password hash corresponds to the hash that is stored.
 - Verifies the user's role (in this example, administrator rights).
4. Considering validation:
 - Success: Starts a session and takes you to the admin panel.
 - Failure: Returns to the login page after flashing an error message.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Admin login</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
  <div class="container">
    <div class="form-box">
      <h2>Admin login</h2>
      {% with messages = get_flashed_messages() %}
        {% if messages %}
          {% for message in messages %}
            <div class="alert">{{ message }}</div>
          {% endfor %}
        {% endif %}
      {% endwith %}
      <form method="POST" action="{{ url_for('login') }}">
        <div class="input-group">
          <input type="text" name="username" required>
          <label>Username</label>
        </div>
        <div class="input-group">
          <input type="password" name="password" required>
          <label>Password</label>
        </div>
        <button type="submit">Login</button>
      </form>
      <p class="switch-form">Don't have an account? <a href="{{ url_for('register') }}">Register</a></p>
    </div>
  </div>
</body>
</html>

```

Figure 35

```

@app.route('/')
def index():
    if current_user.is_authenticated:
        return redirect(url_for('dashboard'))
    return redirect(url_for('login'))

@app.route('/login', methods=['GET', 'POST'])
def login():
    if current_user.is_authenticated:
        return redirect(url_for('dashboard'))

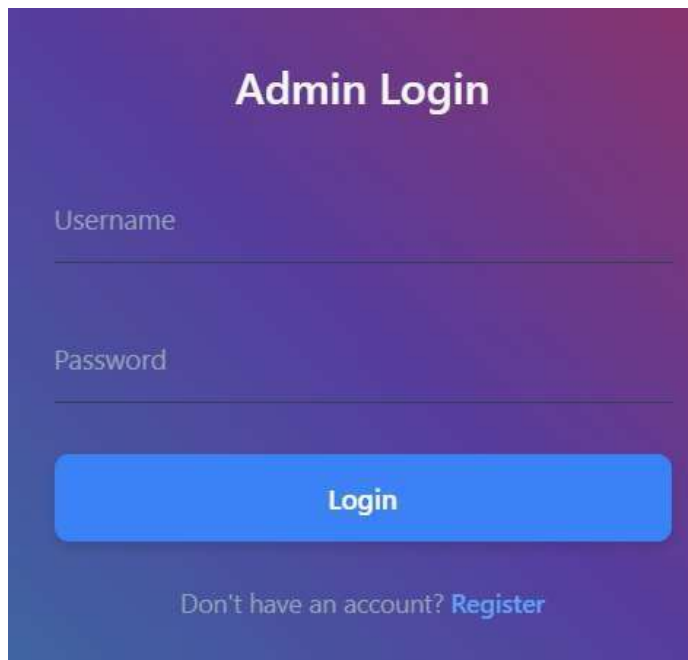
    if request.method == 'POST':
        username = request.form.get('username')
        password = request.form.get('password')

        user_data = users.find_one({'username': username})
        print("user data from DB:", user_data)
        if user_data:
            print("Password in DB:", user_data['password'])
            print("Password entered:", password)
            print("Password check:", check_password_hash(user_data['password'], password))
            if user_data and check_password_hash(user_data['password'], password):
                user = User(user_data)
                login_user(user)
                return redirect(url_for('dashboard'))

        flash('Invalid username or password')
    return render_template('login.html')

```

Figure 34



The image shows a web form titled "Admin Login" on a purple-to-blue gradient background. It contains two input fields: "Username" and "Password", each with a horizontal line for text entry. Below these fields is a blue rectangular button with the word "Login" in white. At the bottom, there is a link that says "Don't have an account? Register", where "Register" is in blue and underlined.

Figure 36

When the user enters the username and password, they will be able to login to the system and view the sales dashboard of Adidas.

Registering page

The registration interface is implemented as an HTML form with the following components:

```

<!-- HTML
html lang="en"
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Admin Registration</title>
<link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
<div class="container">
<div class="form-box">
<h2>Admin Registration</h2>
<% with messages = get_flashed_messages() %>
<% if messages %>
<% for message in messages %>
<div class="alert">{{ message }}</div>
<% endfor %>
<% endif %>
<% endwith %>
<form method="POST" action="{{ url_for('register') }}">
<div class="input-group">
<input type="text" name="username" required>
<label>Username</label>
</div>
<div class="input-group">
<input type="email" name="email" required>
<label>Email</label>
</div>
<div class="input-group">
<input type="password" name="password" required>
<label>Password</label>
</div>
<button type="submit">Register</button>
</form>
<p class="with-form">Already have an account? <a href="{{ url_for('login') }}">Login</a></p>
</div>

```

Figure 37

New administrators can create accounts and use the admin features of the program using the registration system. This section describes the front-end interface and back-end operations used in the registration process.

Key Features:

- Form Fields:
 - Username field (text input with required validation)
 - Email field (email input with built-in browser validation)
 - Password field (password input with required validation)
- User Feedback:
 - Flash message display for success/error notifications
 - Consistent styling with login page for unified user experience
- Navigation:
 - Link to login page for existing users
 - Maintains same container and form-box styling as login page

- Backend Registration Process:

The registration functionality is handled by the Flask route in app.py:

```
@app.route('/register', methods=['GET', 'POST'])
def register():
    if current_user.is_authenticated:
        return redirect(url_for('dashboard'))

    if request.method == 'POST':
        username = request.form.get('username')
        email = request.form.get('email')
        password = request.form.get('password')

        if users.find_one({'$or': [{'username': username}, {'email': email}]}):
            flash('Username or email already exists')
            return redirect(url_for('register'))

        hashed_password = generate_password_hash(password)
        user_data = {
            'username': username,
            'email': email,
            'password': hashed_password
        }
        users.insert_one(user_data)
        flash('Registration successful! Please login.')
        return redirect(url_for('login'))

    return render_template('register.html')
```

Figure 38

- Registration Workflow:
 1. Initial Checks:
 - Redirects authenticated users to dashboard
 - Only processes POST requests for actual registration attempts
 2. Data Validation:
 - Checks for existing username or email using MongoDB query
 - Returns error message if credentials are already taken
 3. Secure Password Handling:
 - Uses Werkzeug's generate_password_hash to create secure password hashes
 - Implements industry-standard hashing (PBKDF2 with SHA256 by default)
 4. User Creation:

- Stores new user document in MongoDB collection
- Includes username, email, and hashed password
- Provides success feedback and redirects to login page

- Security Considerations

1. Password Security:

- Never stores plain text passwords
- Uses salt+hash mechanism for password storage
- Implements work factor to slow down brute force attacks

2. Data Validation:

- Client-side validation via HTML5 attributes (required, email type)
- Server-side validation for duplicate credentials
- MongoDB query prevents duplicate usernames/emails

3. Session Management:

- Prevents authenticated users from accessing registration page
- Follows principle of least privilege

4. Database Operations:

- Uses PyMongo for secure database interactions
- Properly handles ObjectId conversions for MongoDB documents

- Integration with User Authentication System

The registration process works in conjunction with the Flask-Login system:

1. User Model:

- The User class implements Flask-Login's UserMixin
- Stores user ID, username, and email for session management

```
class User(UserMixin):
    def __init__(self, user_data):
        self.id = str(user_data['_id'])
        self.username = user_data['username']
        self.email = user_data['email']
```

Figure 39

2. User Loader:

- load_user function retrieves user data from MongoDB
- Converts MongoDB's ObjectId to string for session handling

```
@login_manager.user_loader
def load_user(user_id):
    try:
        user_data = users.find_one({'_id': ObjectId(user_id)})
        if user_data:
            return User(user_data)
    except Exception as e:
        print("User loader error:", e)
    return None
```

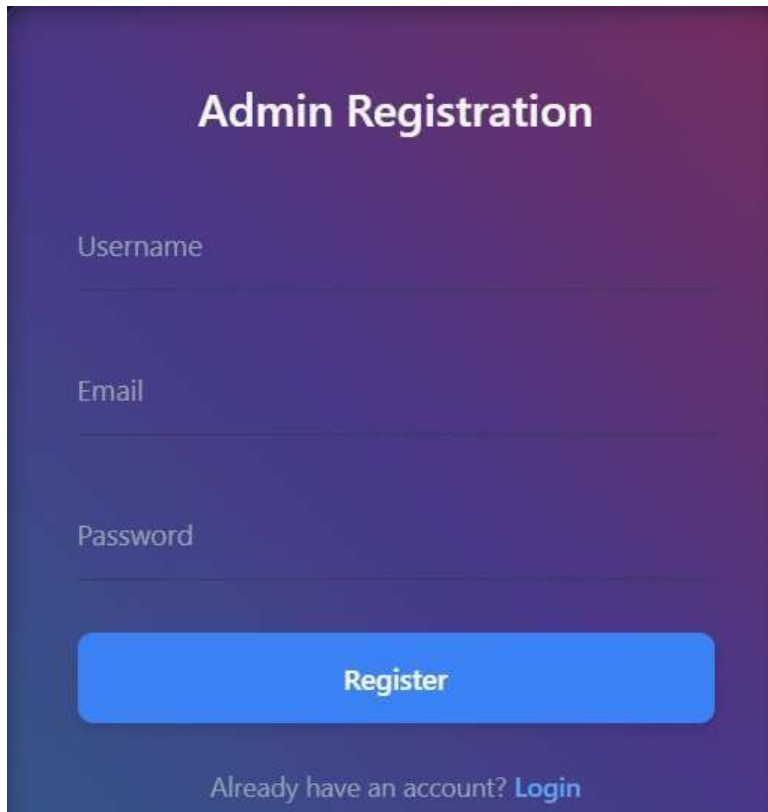
Figure 40

3. Seamless Transition:

- Successful registration redirects to login page
- Consistent user experience between registration and login flows

```
@app.route('/register', methods=['GET', 'POST'])
def register():
    if current_user.is_authenticated:
        return redirect(url_for('dashboard'))
```

Figure 41

The image shows a web form titled "Admin Registration" on a dark purple background. It contains three input fields: "Username", "Email", and "Password", each with a light blue underline. Below these fields is a prominent blue button with the text "Register" in white. At the bottom of the form, there is a link that says "Already have an account? Login".

Admin Registration

Username

Email

Password

Register

Already have an account? [Login](#)

Figure 42

The user will be able to register using their email and password along with a username and successfully register and be able to log into the system to access the dashboard.

Together, these elements provide a comprehensive authentication system where:

- New users can safely signup.
- MongoDB appropriately stores user data.
- Sessions are managed with Flask-Login.
- Security is maintained throughout the flow by the system.

MongoDB in the dashboard web app

The system uses MongoDB, a NoSQL database, to store admin user credentials and authentication data. Here's how it's implemented:

```
# Optional: Set the URI directly if not using .env
MONGODB_URI = os.getenv('MONGODB_URI', 'mongodb+srv://senash:1234@cluster0.mongodb.net/?retryWrites=true&majority=&appName=cluster0')

# MongoDB connection
client = MongoClient(MONGODB_URI)
db = client['adidas_db']
users = db['admin']
```

Key Features of the MongoDB Implementation:

- Database Name: adidas_db - The main database that stores all application data
- Collections:
 - admin collection - Stores admin user credentials with the following fields:
 - _id: Auto-generated unique ObjectId for each user
 - username: Unique admin username
 - email: Admin email address
 - password: Hashed password (for security)
- Security Measures:
 - Passwords are never stored in plain text - they are hashed using Werkzeug's generate_password_hash
 - Password verification uses check_password_hash for secure comparison
 - Connection uses MongoDB's secure URI with retryWrites enabled

User Registration Flow (Storing in MongoDB)

1. When a new admin registers through /register route:
 - System checks if username or email already exists in the admin collection
 - If not, creates a new document with:

```
user_data = {
    'username': username,
    'email': email,
    'password': hashed_password
}
users.insert_one(user_data)
```

2. The password is immediately hashed before storage using PBKDF2 with SHA256:

- This ensures even if the database is compromised, passwords remain protected
- The hash includes a salt to prevent rainbow table attacks

Authentication Flow (Querying MongoDB)

During login at /login route:

System queries MongoDB for the username:

```
user_data = users.find_one({'_id': ObjectId(user_id)})
if user_data:
    return User(user_data)
```

If found, verifies password against the stored hash:

```
if user_data and check_password_hash(user_data['password'], password):
    user = User(user_data)
    login_user(user)
    return redirect(url_for('dashboard'))
```

On successful login:

- Creates a User object with data from MongoDB
- Establishes a session using Flask-Login



Advantages of MongoDB

- Flexible Schema:

- Easy to add new fields to user profiles without migrations
- Can store different data types natively
- Scalability:
 - Handles large numbers of admin users efficiently
 - MongoDB Atlas provides cloud-based scaling options
- Performance:
 - Fast queries for authentication checks
 - Indexing on username/email fields for quick lookups

Dashboard system

The dashboard, which offers dynamic visualizations and business intelligence features, is the primary interface via which administrators examine sales data. This implementation uses Streamlit for the data visualization interface and Flask for authentication.

The below diagram represents part of the dashboard web page:



Figure 43

The dashboard consists of two main components:

1. Flask Application (app.py) - Handles:
 - User authentication (login/registration)
 - Session management

- Routing to the Streamlit dashboard
2. Streamlit Dashboard - Provides:
- Interactive data visualizations
 - Filtering capabilities
 - Data export functionality
 - Predictive analytics
- Flask dashboard integration

The Flask app (app.py) manages access to the dashboard through these key components:

```
@app.route('/dashboard')
@login_required
def dashboard():
    # Start Streamlit in a separate thread if it's not already running
    if not hasattr(app, 'streamlit_thread') or not app.streamlit_thread.is_alive():
        app.streamlit_thread = threading.Thread(target=run_streamlit)
        app.streamlit_thread.daemon = True
        app.streamlit_thread.start()
        # Give Streamlit time to start
        time.sleep(2)
    return redirect('http://localhost:8501')

@app.route('/logout')
@login_required
def logout():
    logout_user()
    return redirect(url_for('login'))

if __name__ == '__main__':
    app.run(debug=True)
```

Figure 44

- Key features of this implementation:
 - Authentication Protection: Uses @login_required decorator to ensure only authenticated users can access
 - Thread Management: Launches Streamlit in a separate daemon thread to avoid blocking the Flask app
 - Seamless Redirect: Automatically redirects users to the Streamlit interface (port 8501)

Dashboard components

The dashboard is organized into several functional sections:

1. Page Navigation

- Provides two main views: Dashboard (default) and Predictive Analysis
- Uses Streamlit's sidebar for persistent navigation

```
# PAGE SELECTOR AT THE TOP
page = st.sidebar.selectbox("Select Page", ["Dashboard", "Predictive Analysis"])
```

Figure 46

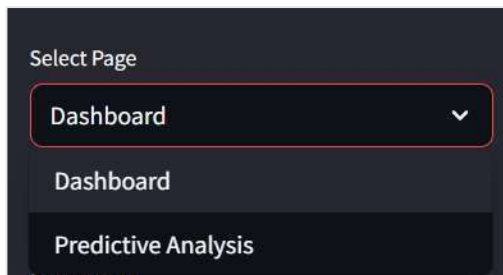


Figure 45

2. Data loading and setup

- Loads sales data from Excel file
- Configures wide layout for optimal visualization space

```
# reading the data from excel file
df = pd.read_excel("Adidas.xlsx")
st.set_page_config(layout="wide")
st.markdown('<style>div.block-container{padding-top:1rem;}</style>', unsafe_allow_html=True)
```

Figure 47

3. Interactive filters

The dashboard includes comprehensive filtering options

Key features:

- Multi-select filters for all major dimensions such as year, month, product, region, retailer.
- Defaults to showing all data
- Real-time filtering of all visualizations

```
st.sidebar.header("Filters")
# Year Filter
years = sorted(df['InvoiceDate'].dt.year.unique())
selected_years = st.sidebar.multiselect(
    "Select Year",
    years,
    default=years
)

# Month Filter
months = sorted(df['InvoiceDate'].dt.month.unique())
month_names = [datetime.date(2020, month, 1).strftime('%B') for month in months]
selected_months = st.sidebar.multiselect(
    "Select Month",
    month_names,
    default=month_names
)

# Product Filter
products = sorted(df['Product'].unique())
selected_products = st.sidebar.multiselect(
    "Select Product",
    products,
    default=products
)
```

```
# Apply filters to the dataframe
filtered_df = df[
    (df['InvoiceDate'].dt.year.isin(selected_years)) &
    (df['InvoiceDate'].dt.strftime('%B').isin(selected_months)) &
    (df['Product'].isin(selected_products)) &
    (df['Retailer'].isin(selected_retailers)) &
    (df['Region'].isin(selected_regions))
]
```

Figure 49

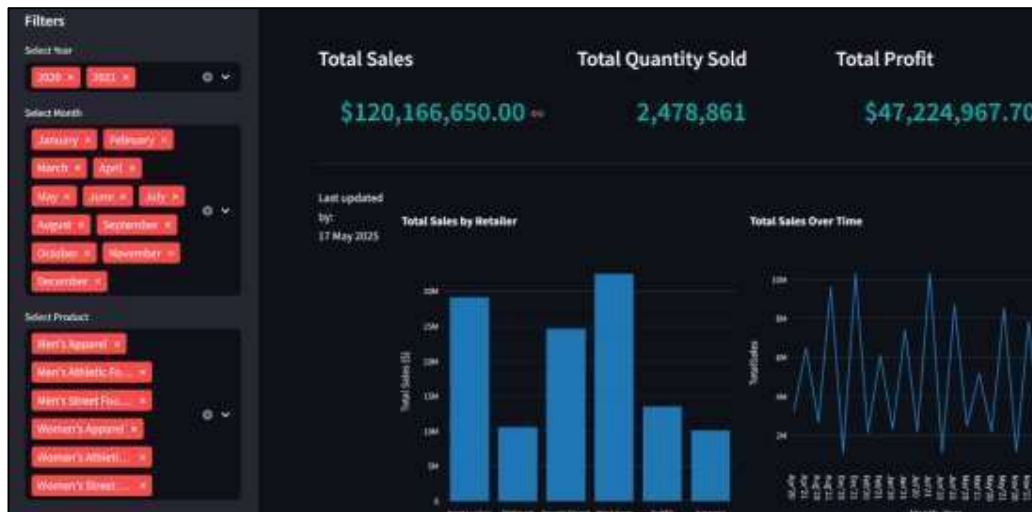


Figure 48

4. Key performance Indicators

Displays three primary metrics:

- Total Sales (currency formatted)
- Total Quantity Sold (formatted)
- Total Profit (currency formatted)

```
# --- KPI METRICS ---
st.markdown("---")
col_kpi1, col_kpi2, col_kpi3 = st.columns(3)

with col_kpi1:
    st.markdown("### Total Sales")
    total_sales = filtered_df['TotalSales'].sum()
    st.markdown(f"<h2 style='text-align: center; color: #00B2A9;'>${total_sales:,.2f}</h2>", unsafe_allow_html=True)

with col_kpi2:
    st.markdown("### Total Quantity Sold")
    total_quantity = filtered_df['UnitsSold'].sum()
    st.markdown(f"<h2 style='text-align: center; color: #00B2A9;'>{total_quantity:,}</h2>", unsafe_allow_html=True)

with col_kpi3:
    st.markdown("### Total Profit")
    total_profit = filtered_df['OperatingProfit'].sum()
    st.markdown(f"<h2 style='text-align: center; color: #00B2A9;'>${total_profit:,.2f}</h2>", unsafe_allow_html=True)

st.markdown("---")
```

Figure 52

Total Sales	Total Quantity Sold	Total Profit
\$120,166,650.00	2,478,861	\$47,224,967.70

Figure 51

5. Main visualizations

a. Sales by retailer (Bar chart)

Compare revenue made across retailers. Identifies which retailer contribute most sales and profitability.

```
# --- RETAILER SALES CHART ---
with col4:
    fig = px.bar(filtered_df, x="Retailer", y="TotalSales",
                  labels={"TotalSales": "Total Sales ($)"},
                  title="Total Sales by Retailer",
                  hover_data=["TotalSales"],
                  template="gridon", height=500)
    st.plotly_chart(fig, use_container_width=True)
```

Figure 54

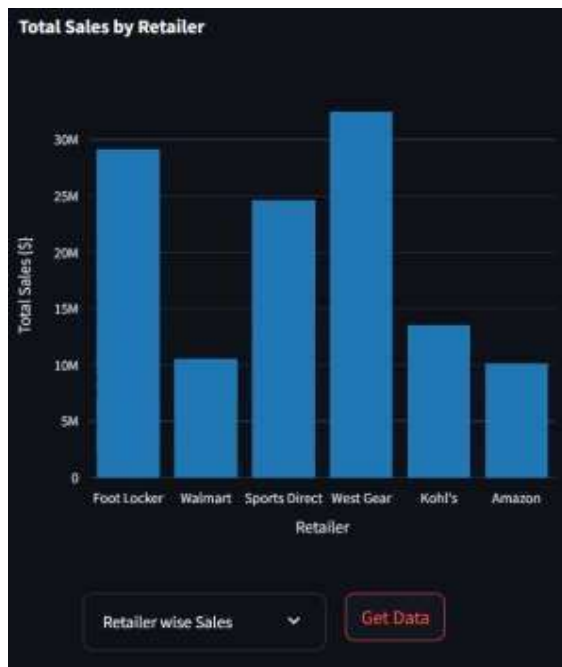


Figure 53

b. Sales over time (Line chart)

Tracks sales over time (InvoiceDate aggregated by month)

```
# --- TIME SERIES CHART ---
filtered_df["Month_Year"] = filtered_df["InvoiceDate"].dt.strftime("%b %y")
result = filtered_df.groupby(by=filtered_df["Month_Year"])[["TotalSales"]].sum().reset_index()

with cols:
    fig1 = px.line(result, x="Month_Year", y="TotalSales",
                  title="Total Sales Over Time",
                  template="gridon")
    st.plotly_chart(fig1, use_container_width=True)

with view2:
    expander = st.expander("Monthly Sales")
    data = result
    expander.write(data)

with dwn2:
    st.download_button("Get Data", data=result.to_csv().encode("utf-8"),
                      file_name="MonthlySales.csv", mime="text/csv")
```

Figure 55



Figure 56

c. Geographic analysis (Treemap)

Visualizes regional wise sales, hierarchical view of sales by Region > City. Identifies which products contribute most number of sales.

```
# --- Treemap ---
col7 = st.columns([0.1,1])
treemap = filtered_df[["Region","City","TotalSales"]].groupby(by=["Region","City"])["TotalSales"].sum().reset_index()

def format_sales(value):
    if value >= 0:
        return ':{.2f} Lakh'.format(value / 1_000_00)

treemap["TotalSales (Formatted)"] = treemap["TotalSales"].apply(format_sales)

fig4 = px.treemap(treemap, path=["Region","City"], values="TotalSales",
                  hover_name="TotalSales (Formatted)",
                  hover_data=["TotalSales (Formatted)"],
                  color="City", height=700, width=600)
fig4.update_traces(textinfo="label+value")

with col7:
    st.subheader("point right: Total Sales by Region and City in Treemap")
    st.plotly_chart(fig4, use_container_width=True)

_, view4, dm4 = st.columns([0.5,0.45,0.45])
with view4:
    result2 = filtered_df[["Region","City","TotalSales"]].groupby(by=["Region","City"])["TotalSales"].sum()
    expander = st.expander("View data for Total Sales by Region and City")
    expander.write(result2)
with dm4:
    st.download_button("Get Data", data=result2.to_csv().encode("utf-8"),
                      file_name="Sales_by_Region.csv", mime="text/csv")
```

Figure 58

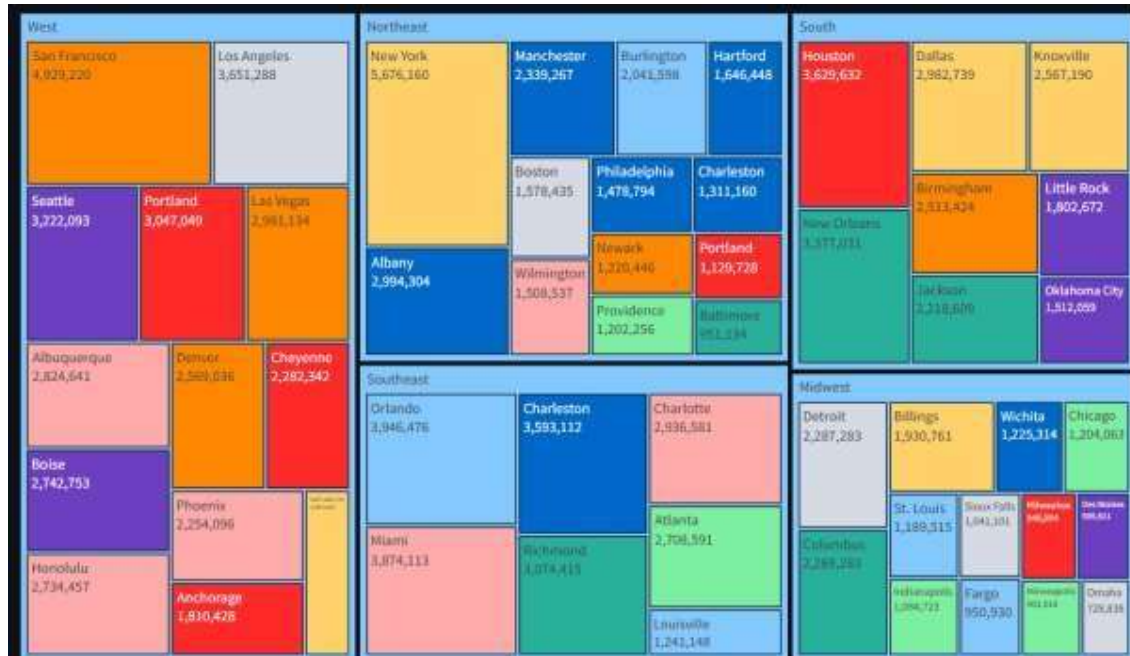


Figure 57

d. Product performance analysis (scatter plot)

Product performance: Sales vs Unit sold visualizes the performance of the product by the total units sold and the total sales made

```
# 1. Product performance analysis
col8, col9 = st.columns(2)
with col8:
    product_performance = filtered_df.groupby('Product')[['TotalSales', 'UnitsSold']].sum().reset_index()
    fig5 = px.scatter(product_performance,
                      x='UnitsSold',
                      y='TotalSales',
                      color='Product',
                      size='TotalSales',
                      title='Product Performance: Sales vs Units Sold',
                      labels={'UnitsSold': 'Units Sold', 'TotalSales': 'Total Sales ($)'},
                      template='gridon')
    st.plotly_chart(fig5, use_container_width=True)
```

Figure 60



Figure 59

e. Additional visualizations

Sales distribution by method (Pie chart) visualizes the number of sales made by each distribution method. Identifies which sales distribution method contribute most sales.

Operating Margin by product (%) compare profitability across products to identify high/lowmargin offerings. Identifies which products contribute most to profitability.

Top 10 products by revenue visualizes the top 10 highest products that made the highest number of sales for Adidas in the given time period.

```
# 2. Sales Method Distribution
with col9:
    sales_method = filtered_df.groupby("SalesMethod")["TotalSales"].sum().reset_index()
    fig6 = px.pie(sales_method,
                  values='TotalSales',
                  names='SalesMethod',
                  title='Sales Distribution by Method',
                  hole=0.4)
    st.plotly_chart(fig6, use_container_width=True)

# 3. Profit Margin Analysis
col10, col11 = st.columns(2)
with col10:
    profit_margin = filtered_df.groupby("Product")["TotalSales", "OperatingMargin"].sum().reset_index()
    fig7 = px.bar(profit_margin,
                  x='Product',
                  y='OperatingMargin',
                  title='Operating Margin by Product (%)',
                  template='gridon')
    st.plotly_chart(fig7, use_container_width=True)

# 4. Top 10 Products by Revenue
with col11:
    top_products = filtered_df.groupby("Product")["TotalSales"].sum().sort_values(ascending=False).head(10)
    fig8 = px.bar(top_products.reset_index(),
                  x='TotalSales',
                  y='Product',
                  title='Top 10 Products by Revenue',
                  orientation='h',
                  template='gridon')
    st.plotly_chart(fig8, use_container_width=True)

st.divider()
```

Figure 62

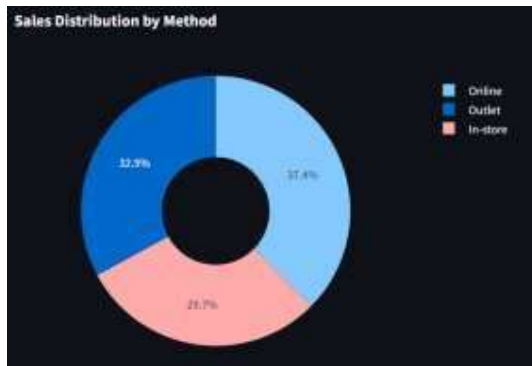


Figure 61

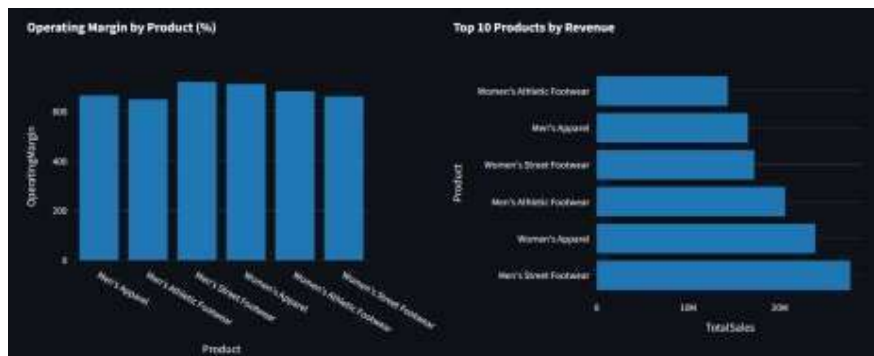


Figure 63

Total sales and units sold by state represents the number of sales and units sold in each state. Identifies which state contribute most to sales and which stat doesn't.

```
# --- STATE-WISE SALES AND UNITS SOLD ---
result1 = filtered_df.groupby(by="State")[["TotalSales","UnitsSold"]].sum().reset_index()

fig3 = go.Figure()
fig3.add_trace(go.Bar(x=result1["State"], y=result1["TotalSales"], name="Total Sales"))
fig3.add_trace(go.Scatter(x=result1["State"], y=result1["UnitsSold"], mode="lines",
                          name="Units Sold", yaxis="y2"))
fig3.update_layout(
    title="Total Sales and Units Sold by State",
    xaxis=dict(title="State"),
    yaxis=dict(title="Total Sales", showgrid=False),
    yaxis2=dict(title="Units Sold", overlaying="y", side="right"),
    template="gridon",
    legend=dict(x=1, y=1.1)
)

_, col6 = st.columns([0.1,1])
with col6:
    st.plotly_chart(fig3, use_container_width=True)

_, view3, dwn3 = st.columns([0.5,0.45,0.45])
with view3:
    expander = st.expander("View Data for Sales by Units Sold")
    expander.write(result1)
with dwn3:
    st.download_button("Get Data", data=result1.to_csv().encode("utf-8"),
                      file_name="Sales_by_UnitsSold.csv", mime="text/csv")

st.divider()
```

Figure 64

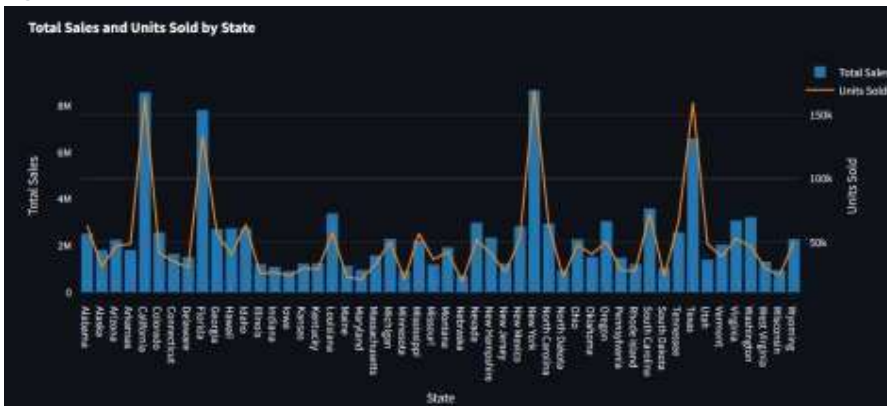


Figure 65

6. Data export functionality

Provides export options to export and download as csv files for:

- Adidas sales data
- Retailer sales data
- Monthly trends
- Regional breakdowns ○ Raw filtered data

```
# --- DOWNLOAD BUTTONS AND EXPANDERS ---
_, view1, dwn1, view2, dwn2 = st.columns([0.15,0.20,0.20,0.20,0.20])
with view1:
    expander = st.expander("Retailer wise Sales")
    data = filtered_df[["Retailer","TotalSales"]].groupby(by="Retailer")["TotalSales"].sum()
    expander.write(data)
with dwn1:
    st.download_button("Get Data", data=data.to_csv().encode("utf-8"),
                        file_name="RetailerSales.csv", mime="text/csv")
```

Figure 66



Figure 67

7. Summary statistics

Displays real-time calculated metrics in the sidebar based on current filters.

```
# Add summary statistics
st.sidebar.header("Summary Statistics")
st.sidebar.write(f"Total Sales: ${filtered_df['TotalSales'].sum():.2f}")
st.sidebar.write(f"Total Units Sold: {filtered_df['UnitsSold'].sum():,}")
st.sidebar.write(f"Average Operating Margin: {filtered_df['OperatingMargin'].mean():.2f}%")
```

Figure 68

Detailed Margin Analysis

Sales by Margin Range

	MarginRange	TotalSales	SalesPercentage	OperatingMargin
0	0-20%	\$4,320,890.00	3.60%	0.19%
1	20-40%	\$51,505,329.00	42.86%	0.33%
2	40-60%	\$53,060,047.00	44.16%	0.45%
3	60-80%	\$10,567,840.00	8.79%	0.57%
4	80-100%	\$712,544.00	0.59%	0.71%

Key Insights

- Highest sales are in the 40-60% range
- 44.2% of total sales come from the 40-60% range
- Average operating margin across all ranges: 0.4%

Figure 69

Technical implementation details

1. Data Processing:

- Uses Pandas for efficient data manipulation

- Implements dynamic filtering of the DataFrame based on user selections ○ Includes date/time formatting for proper visualization

2. Visualization:

- Leverages Plotly Express for interactive charts
- Uses Plotly Graph Objects for more complex visualizations ○ Implements consistent theming ("gridon" template)

3. Layout Management:

- Uses Streamlit columns (st.columns) for responsive layout
- Implements expanders for optional detailed views ○ Maintains consistent spacing and dividers

4. Performance Considerations:

- Efficient DataFrame operations minimize processing time
- Cached data loading (though not shown here, could be added) ○ Threaded Streamlit launch prevents Flask app blocking

Security Integration

The dashboard maintains security through:

- Flask-Login protected routes
- Session-based authentication
- Secure redirects between Flask and Streamlit
- No sensitive data exposure in the frontend

This implementation provides administrators with a powerful, interactive tool for sales analysis while maintaining strict access control and data security.

Predictive analytics page

The dashboard includes two complementary predictive analysis pages that provide different approaches to sales forecasting:

- Predictive Analysis (Prophet): Uses Facebook's Prophet time series forecasting model
- Advanced Predictive Models (Random Forest): Uses Random Forest regression with feature engineering

Both pages maintain consistent data processing, visualization styles, and evaluation metrics for fair comparison while offering different modeling approaches.

Based on past sales and customer data from the Adidas dataset, the Predictive Analysis page in the Adidas Dashboard web app is intended to offer insights into the future. This page makes strategic decisions for marketing, sales optimization, and inventory management by using machine learning approaches to predict future sales patterns.

Prophet Model

Facebook (now Meta) created the Prophet model, a time series forecasting technique. In order to generate forecasts, this additive model breaks down time series data into components related to trend, seasonality, and holidays. Prophet is renowned for its capacity to manage seasonality (daily, weekly, and annual), non-linear trends, and the impact of holidays.

Time series analysis is a statistical approach that entails gathering data at consistent intervals to recognize patterns and trends. This methodology is employed for making well-informed decisions and precise forecasts by leveraging insights derived from historical data (GeeksforGeeks, 2024).

Key components of the Model Important

attributes and elements:

- Additive Model:

To generate the forecast, the prophecy breaks down the time series into constituent parts.

- The trend component:

The trend component models the data's general long-term flow. For the trend, Prophet may choose a logistic or piecewise linear growth curve.

- Seasonal Components:

Documents recurring trends, including daily, weekly, or annual seasonality, that take place within the time series. It models weekly seasonality using dummy variables and annual seasonality using Fourier series.

- Ease of Use:

Prophet is designed to be relatively easy to use, even for users with limited statistical knowledge.

- Handling of Missing Data and Outliers:

Prophet can handle missing data and outliers in the time series.

- Automatic Seasonality Detection:

Prophet can automatically detect yearly, weekly, and daily seasonality patterns.

How it Works:

1. Decomposition:

Prophet breaks down the time series data into trend, seasonality, and holiday components.

2. Model Fitting:

It fits a model to each component, using different techniques for trend, seasonality, and holidays.

3. Prediction:

It combines the predictions of the individual components to produce the final forecast.

User Interface Elements

- Page Title and Description

The page is titled "Predictive Analysis: Future Sales Forecast" using `st.title()`.

A horizontal line (`st.markdown("---")`) separates the title from the content.

An info box (st.info()) explains the purpose: forecasting future sales with Facebook Prophet based on historical data.

```
elif page == "Predictive Analysis":
    st.title("Predictive Analysis: Future Sales Forecast")
    st.markdown("---")
    st.info("This section uses Facebook Prophet to forecast future sales based on historical data.")
```

Figure 70

- Side bar controls

The sidebar allows users to customize the forecasting:

- Metric to Forecast:

Select from "Total Sales", "Units Sold", "Operating Profit", and "Operating Margin".
(st.sidebar.selectbox)

- Aggregation Level:

Choose the level of aggregation for forecasting:

- ☐ Overall (all data combined)
- ☐ By Product ☐ By Region

Aggregation Level	What It Groups	Example Use
By Region	Aggregates total sales per region	See how each region is performing (e.g., Europe vs Asia)
By Product	Aggregates total sales per product category	See which product categories are trending (e.g., Shoes vs Hoodies)

- Forecast Horizon:

Number of months to predict into the future (3, 6, or 12 months).

- Confidence Interval:

Slider to select the confidence interval percentage for prediction uncertainty (70% to 95%).

70%: Narrower range → less uncertainty shown. May miss more outliers. Less conservative.

80% (default in many Prophet use-cases): A good trade-off between accuracy and clarity.

90%-95%: Wider range → more uncertainty captured. More conservative but may appear too broad, especially if data is noisy.

Best performing interval would be to use 80% confidence interval for sales forecasting and general business use — it balances predictive power with reasonable uncertainty.

Each control has a unique key to preserve state.

```
# --- SIDEBAR CONTROLS ---
st.sidebar.header("Forecast Settings")

# Metric selection
forecast_metric = st.sidebar.selectbox(
    "Select Metric to Forecast",
    ["Total Sales", "Units Sold", "Operating Profit", "Operating Margin"],
    index=0,
    key="metric_select" # Unique key added
)

# Aggregation level
agg_level = st.sidebar.selectbox(
    "Aggregation Level",
    ["Overall", "By Product", "By Region"],
    index=0,
    key="agg_level_select" # Unique key added
)

# Forecast horizon
periods = st.sidebar.selectbox(
    "Forecast Horizon (months)",
    [3, 6, 12],
    index=1,
    key="horizon_select" # Unique key added
)

# Confidence interval
confidence_level = st.sidebar.slider(
    "Confidence Interval",
    70, 95, 80,
    key="confidence_slider" # Unique key added
)
```

Figure 71

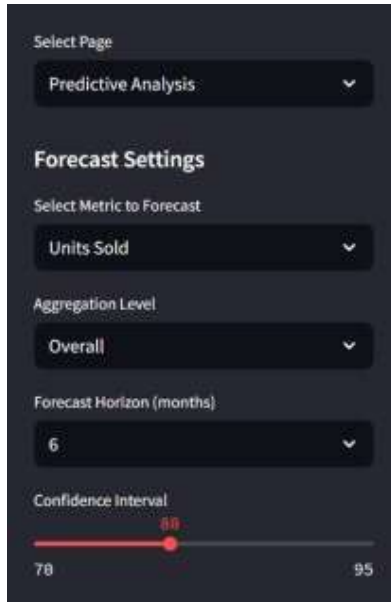


Figure 72

- Visualization Area:

Interactive forecast plots showing historical data and future predictions

Data tables displaying numerical forecast values

Model evaluation metrics and comparison tools

- Overall Forecast:

Plots historical values, forecasted values, and confidence intervals using Plotly line charts.

Historical data shown as blue lines+markers.

Forecast as green lines.

Confidence intervals as shaded areas.

- Grouped Forecasts (By Product/Region):

Creates tabs for each group, each showing a similar Plotly chart with historical + forecast + confidence interval.

The y-axis label and formatting for numbers vary based on the selected metric (currency, units, percentage).

Displays a forecast table with predicted values and confidence intervals formatted nicely (currency with \$, percentages with %, or integers).

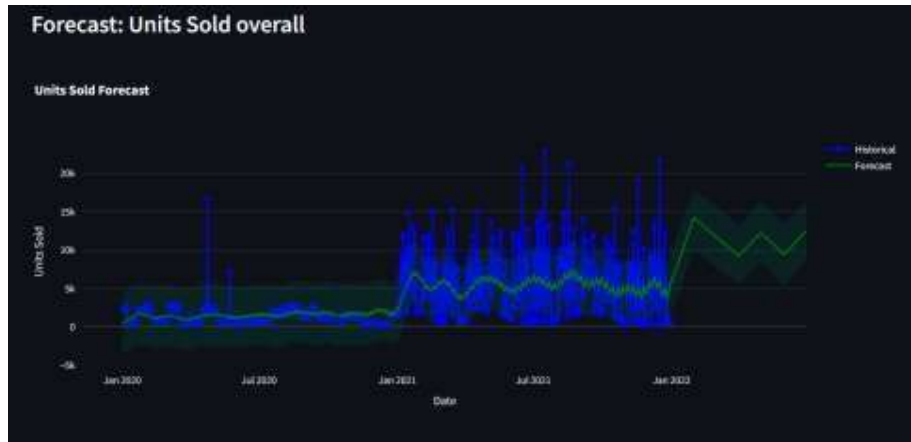


Figure 73



Figure 74



Figure 77

```
# Create comparison plot
if compare_groups:
    fig_compare = go.Figure()

    for group in compare_groups:
        if agg_level == "Overall":
            forecast = forecast_result
        else:
            forecast = forecast_result[group]

        # Filter to only future period
        future_dates = forecast['ds'] > forecast_data['ds'].max()
        future_forecast = forecast[future_dates]

        fig_compare.add_trace(go.Scatter(
            x=future_forecast['ds'],
            y=future_forecast['yhat'],
            mode='lines+markers',
            name=group,
            hovertemplate=f"{group}<br>Date: %({x})%<br>Predicted {compare_metric}: %({y})"
        )))
```

Figure 76

```
# Plot actual vs predicted
fig_eval = go.Figure()
fig_eval.add_trace(go.Scatter(
    x=actual_test['ds'],
    y=actual_test['y'],
    mode='lines+markers',
    name='Actual'
)))
fig_eval.add_trace(go.Scatter(
    x=pred_test['ds'],
    y=pred_test['yhat'],
    mode='lines+markers',
    name='Predicted'
)))
fig_eval.update_layout(
    title=f"Actual vs Predicted {forecast_metric} (Test Set)",
    xaxis_title="Date",
    yaxis_title=forecast_metric,
    template="gridon"
)
st.plotly_chart(fig_eval, use_container_width=True)
```

Figure 75

Technical Implementation

- Data Preparation (prepare_forecast_data)

Converts the date column (InvoiceDate) to datetime format.

Maps the user-selected metric to the corresponding data column (e.g., "Total Sales" → "TotalSales").

Depending on the aggregation level, it:

- Groups data by date (Overall), or
- Groups by date + Product/Region.

The output is a DataFrame suitable for Prophet with columns renamed to "ds" (date) and "y" (target metric), and optionally "group" for grouped forecasts.

```
# --- DATA PREPARATION ---
def prepare_forecast_data(df, metric, groupby=None):
    """Prepare data for Prophet forecasting"""
    df = df.copy()

    # Convert InvoiceDate to datetime if not already
    df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])

    # Select the appropriate metric column
    metric_col = {
        "Total Sales": "TotalSales",
        "Units sold": "UnitsSold",
        "Operating Profit": "OperatingProfit",
        "Operating Margin": "OperatingMargin"
    }[metric]

    if groupby:
        # Group by date and the specified column
        forecast_df = df.groupby(['InvoiceDate', groupby])[metric_col].sum().reset_index()
        forecast_df = forecast_df.rename(columns={"InvoiceDate": "ds", metric_col: "y", groupby: "group"})
    else:
        # Aggregate by date only
        forecast_df = df.groupby('InvoiceDate')[metric_col].sum().reset_index()
        forecast_df = forecast_df.rename(columns={"InvoiceDate": "ds", metric_col: "y"})

    return forecast_df
```

Figure 78

- Running the Forecast (run_prophet_forecast)

If grouped by Product or Region, it runs a separate Prophet model for each group.

It selects the seasonality mode based on whether the metric can be zero or negative:

- additive for metrics with zero or negative values
- multiplicative for strictly positive metrics

Fits Prophet models and generates future forecasts for the chosen horizon.

Returns forecast results:

- A dictionary of forecasts for each group, or
- A single forecast DataFrame for overall data.

• Forecast Execution

Call `prepare_forecast_data` with the user's selections.

Calls `run_prophet_forecast` with the prepared data and forecast settings.

Stores the forecast results for visualization.

```
# --- FORECASTING ---
def run_prophet_forecast(data, periods, confidence_level, groupby=None):
    """Run Prophet forecast on the given data"""
    if groupby:
        # For grouped forecasts
        groups = data['group'].unique()
        forecasts = {}

        for group in groups:
            group_data = data[data['group'] == group][['ds', 'y']]

            # Handle zero values for multiplicative seasonality
            if group_data['y'].min() <= 0:
                # Additive seasonality for metrics that can be zero or negative
                m = Prophet(seasonality_mode='additive', interval_width=confidence_level/100)
            else:
                # Multiplicative seasonality for strictly positive metrics
                m = Prophet(seasonality_mode='multiplicative', interval_width=confidence_level/100)

            m.fit(group_data)
            future = m.make_future_dataframe(periods=periods, freq='M')
            forecast = m.predict(future)
            forecasts[group] = forecast

        return forecasts
    else:
        # For overall forecast
        if data['y'].min() <= 0:
            m = Prophet(seasonality_mode='additive', interval_width=confidence_level/100)
        else:
            m = Prophet(seasonality_mode='multiplicative', interval_width=confidence_level/100)

        m.fit(data)
        future = m.make_future_dataframe(periods=periods, freq='M')
        forecast = m.predict(future)
```

Figure 80

```
# Show forecast table
st.subheader("Forecast Data")
forecast_display = forecast_result[['ds', 'yhat', 'yhat_lower', 'yhat_upper']].tail(periods)
forecast_display = forecast_display.rename(columns={
    'ds': 'Date',
    'yhat': f'Predicted {forecast_metric}',
    'yhat_lower': 'Lower Estimate',
    'yhat_upper': 'Upper Estimate'
})
```

Figure 79

Forecast Data				
	Date	Predicted Units Sold	Lower Estimate	Upper Estimate
0	Jan 2022	14,235	9,979	17,961
1	Feb 2022	11,856	7,751	15,911
2	Mar 2022	9,214	5,561	13,135
3	Apr 2022	12,255	8,380	16,376
4	May 2022	9,417	5,337	13,394
5	Jun 2022	12,568	8,691	16,423

Figure 81

Model Evaluation

Implements a simple train/test split (last 3 months as test set).

Trains Prophet on the training data and predicts for the test period.

Calculates evaluation metrics for each filter (total sales, units sold, profits etc.):

- MAE (Mean Absolute Error)
- RMSE (Root Mean Squared Error) □ MAPE (Mean Absolute Percentage Error)

Displays the evaluation metrics in a table.

Shows a comparison plot between actual test values and predicted values for the test period.

Performance Metrics:

- MAE (Mean Absolute Error): Average absolute difference between predicted and actual values
- RMSE (Root Mean Squared Error): Penalizes larger errors more heavily
- MAPE (Mean Absolute Percentage Error): Percentage error for relative comparison

Evaluation Process:

- Splits historical data into train/test sets (3-month holdout)
- Trains model on historical data
- Compares predictions against actual values
- Calculates and displays metrics

```
# --- MODEL EVALUATION ---
st.markdown("---")
st.subheader("📊: Model Evaluation")

# Create train/test split
test_size = 3 # months
eval_results = []

if agg_level == "Overall":
    # For overall forecast evaluation
    monthly_data = forecast_data.copy()
    monthly_data['YearMonth'] = monthly_data['ds'].dt.to_period('M')
    monthly_data = monthly_data.groupby('YearMonth')['y'].sum().reset_index()
    monthly_data['ds'] = monthly_data['YearMonth'].dt.to_timestamp()
    monthly_data = monthly_data[['ds', 'y']]

    train_df = monthly_data.iloc[:-test_size]
    test_df = monthly_data.iloc[-test_size:]

    # Train model
    if monthly_data['y'].min() <= 0:
        m = Prophet(seasonality_mode='additive')
    else:
        m = Prophet(seasonality_mode='multiplicative')

    m.fit(train_df)

    # Make future dataframe for test period
    future = m.make_future_dataframe(periods=test_size, freq='M')
    forecast = m.predict(future)

    # Get test predictions
    pred_test = forecast[['ds', 'yhat']].tail(test_size).reset_index(drop=True)
    actual_test = test_df[['ds', 'y']].reset_index(drop=True)
```

Figure 82

```
# Make future dataframe for test period
future = m.make_future_dataframe(periods=test_size, freq='M')
forecast = m.predict(future)

# Get test predictions
pred_test = forecast[['ds', 'yhat']].tail(test_size).reset_index(drop=True)
actual_test = test_df[['ds', 'y']].reset_index(drop=True)

# Calculate metrics
mae = mean_absolute_error(actual_test['y'], pred_test['yhat'])
rmse = np.sqrt(mean_squared_error(actual_test['y'], pred_test['yhat']))
mape = np.mean(np.abs((actual_test['y'] - pred_test['yhat']) / actual_test['y'])) * 100

# Store results
eval_results.append([
    "Group": "Overall",
    "MAE": mae,
    "RMSE": rmse,
    "MAPE": mape
])

# Show comparison
st.write(f"***Evaluation for Overall (forecast_metric):***")
col1, col2, col3 = st.columns(3)
col1.metric("MAE", f"%s(mae:.2f)" % mae if forecast_metric in ["Total Sales", "Operating Profit"] else f"%s(mae:.1f)" % mae if forecast_metric == "Operating Margin" else f"%s(mae:.2f)" % mae)
col2.metric("RMSE", f"%s(rmse:.2f)" % rmse if forecast_metric in ["Total Sales", "Operating Profit"] else f"%s(rmse:.1f)" % rmse if forecast_metric == "Operating Margin" else f"%s(rmse:.2f)" % rmse)
col3.metric("MAPE", f"%s(mape:.2f)" % mape if forecast_metric in ["Total Sales", "Operating Profit"] else f"%s(mape:.1f)" % mape if forecast_metric == "Operating Margin" else f"%s(mape:.2f)" % mape)
```

Figure 83



Figure 84

Business Value Features

- Forecast Comparison Tool:

Side-by-side comparison of different products/regions

Helps identify high-growth opportunities

Supports decision-making for resource allocation

```
# --- FORECAST COMPARISON TOOL ---
st.markdown("----")
st.subheader("Forecast Comparison Tool")

col1, col2 = st.columns(2)

with col1:
    compare_metric = st.selectbox(
        "Select Metric to Compare",
        ["Total Sales", "Units Sold", "Operating Profit", "Operating Margin"],
        index=0,
        key="compare_metric_select" # unique key added
    )

with col2:
    compare_groups = st.multiselect(
        f"Select {agg_level.split('.')[1]} to Compare" if agg_level != "Overall" else "Select to Compare",
        list(forecast_result.keys()) if agg_level != "Overall" else ["Overall"],
        default=list(forecast_result.keys())[:2] if agg_level != "Overall" else ["Overall"],
        key="compare_groups_select" # unique key added
    )

# Create comparison plot
if compare_groups:
    fig_compare = go.Figure()

    for group in compare_groups:
        if agg_level == "Overall":
            forecast = forecast_result
        else:
            forecast = forecast_result[group]

        # Filter to only future period
        future_dates = forecast["ds"] > forecast_data["ds"].max()
```

Figure 85

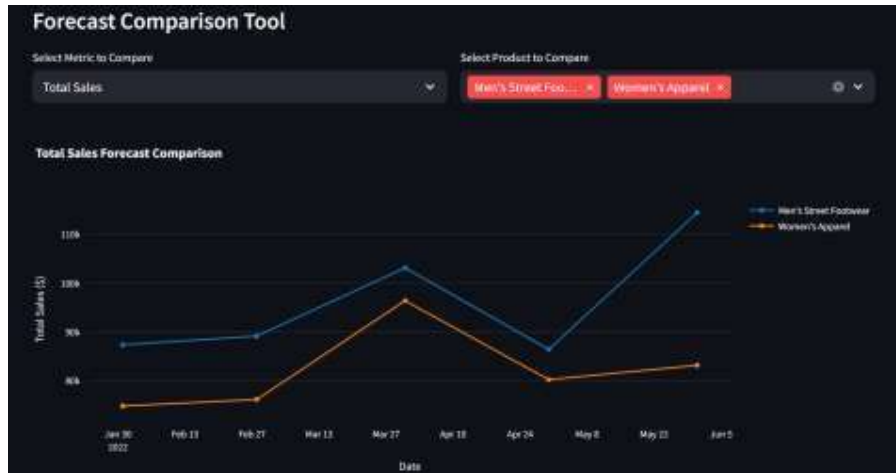


Figure 86

- Data Export:

CSV download capability for all forecast results

Enables further analysis in other tools

Facilitates reporting and presentation

```
if st.button("Prepare forecast data for download", key="download_btn"):
    if agg_level == "Overall":
        forecast_download = forecast_result[['ds', 'yhat', 'yhat_lower', 'yhat_upper']]
        forecast_download = forecast_download.rename(columns={
            'ds': 'date',
            'yhat': f'predicted ({forecast_metric})',
            'yhat_lower': f'lower estimate',
            'yhat_upper': f'upper estimate'
        })
    else:
        forecast_download = pd.DataFrame()

    for group, forecast in forecast_result.items():
        group_forecast = forecast[['ds', 'yhat', 'yhat_lower', 'yhat_upper']]
        group_forecast['group'] = group
        forecast_download = pd.concat([forecast_download, group_forecast])

    forecast_download = forecast_download.rename(columns={
        'ds': 'date',
        'yhat': f'predicted ({forecast_metric})',
        'yhat_lower': f'lower estimate',
        'yhat_upper': f'upper estimate'
    })

    # Convert to CSV
    csv = forecast_download.to_csv(index=False).encode("utf-8")

    # Download button
    st.download_button(
        label="Download Forecast Data",
        data=csv,
        file_name=f"{forecast_metric.replace(' ', '_')}_forecast_{agg_level.replace(' ', '_')}.csv",
        mime="text/csv",
        key=f"final_download_btn" + unique_key_added
    )
```

Figure 87



Figure 88

Technical Considerations

1. Seasonality Management:

- Automatically alternates between multiplicative and additive seasonality according to the properties of the data.
- Guarantees that zero or negative values for operating profit or margin are handled correctly.

2. Enhancement of Performance:

- Forecast findings were cached to increase responsiveness.
- Effective data collection prior to modelling
- Processing grouped forecasts in parallel

3. Experience of the User:

- Clearly visible differences between the projected and historical periods
- Interactive tooltips for in-depth analysis of data
- Design that adapts to various screen sizes

Business Applications

- Inventory Planning: Unit sales forecasts aid in stock level optimization
- Revenue Projections: Financial planning is supported by sales projections.
- Performance Monitoring: Metrics from operations reveal areas that require improvement.
- Scenario Analysis: Risk evaluation is provided via confidence intervals

This implementation offers a complete forecasting solution that blends business-friendly visualization and interaction features with statistical rigor.

Advance Predictive Analysis Page

The Advanced Predictive Analysis page implements a Random Forest Regressor model for time series forecasting, providing an alternative machine learning approach to the Prophet model. This page maintains consistent data processing and visualization with the Prophet implementation while offering different modeling capabilities.

Random Forest Regressor Model

An ensemble of decision trees is used by the supervised learning algorithm Random Forest to forecast results. Using a method known as bagging, it generates several decision trees from arbitrary subsets of the data.

The model generates a final result by combining the predictions made by each individual tree. By mixing the outputs of many trees, random forest reduces variance and improves accuracy in regression tasks by predicting continuous target variables. Compared to individual decision trees, this method enables it to handle larger datasets and capture more intricate relationships (AnalytixLabs, 2023).

Key features of random forests:

- Speed: Efficient and fast, processing large datasets with fewer parameters than other ensemble methods.
- Accuracy: High accuracy due to individualized decision trees that classify data points based on multiple features.
- Feature Selection: Selects the most important features to improve accuracy by measuring each feature's contribution.
- Automation: Easily automated for quick and reliable outcomes.
- Robustness: Resilient to noise and errors, suitable for real-world datasets with gaps or corrupted data.
- Low Overfitting Risk: Reduces overfitting by using independent decision trees, improving generalization.

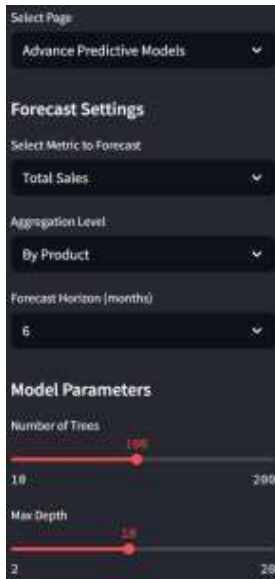
Key Components of the page

1. User Interface Elements

- Main Controls (Sidebar):
 - Metric Selection: Same options as Prophet (Total Sales, Units Sold, etc.)
 - Aggregation Level: Consistent with Prophet (Overall, By Product, By Region)
 - Forecast Horizon: Same 3/6/12 month options
 - Model Parameters:
 - Unique to Random Forest:
 - Number of Trees (10-200)
 - Max Depth (2-20)

```
elif page == "Advance Predictive Models":
    st.title("Advanced Predictive Analysis: Random Forest Forecasting")
    st.markdown("----")
    st.info("This section uses Random Forest Regressor to forecast future sales based on historical data, with consistent data processing as the Prophet model.")
```

```
# Model parameters
st.sidebar.header("Model Parameters")
n_estimators = st.sidebar.slider("Number of Trees", 10, 200, 100)
max_depth = st.sidebar.slider("Max Depth", 2, 20, 10)
```



Select Page

Advance Predictive Models

Forecast Settings

Select Metric to Forecast

Total Sales

Aggregation Level

By Product

Forecast Horizon (months)

6

Model Parameters

Number of Trees

10 100 200

Max Depth

2 10 20

- Visualization Area:
 - Maintains same visual style as Prophet for consistency
 - Adds feature importance visualization
 - Same interactive elements (hover tooltips, etc.)



- Download forecasts feature

Allows you to download a csv file of the forecasts

```
# --- DOWNLOAD FORECASTS ---
st.markdown("---")
st.subheader("Download Forecast Data")

if st.button("Prepare Forecast Data for Download", key="download_btn_rf"):
    if agg_level == "Overall":
        forecast_download = forecast_result[['ds', 'yhat', 'yhat_lower', 'yhat_upper']]
        forecast_download = forecast_download.rename(columns={
            'ds': 'Date',
            'yhat': f'Predicted {forecast_metric}',
            'yhat_lower': 'Lower Estimate',
            'yhat_upper': 'Upper Estimate'
        })
    else:
        forecast_download = pd.DataFrame()

    for group, forecast in forecast_result.items():
        group_forecast = forecast[['ds', 'yhat', 'yhat_lower', 'yhat_upper']]
        group_forecast['Group'] = group
        forecast_download = pd.concat([forecast_download, group_forecast])

    forecast_download = forecast_download.rename(columns={
        'ds': 'Date',
        'yhat': f'Predicted {forecast_metric}',
        'yhat_lower': 'Lower Estimate',
        'yhat_upper': 'Upper Estimate'
    })
})
```



Downloaded CSV (Forecasts data) File:

	A	B	C	D	E
1	Date	Predicted_Total Sales	Lower_Estimate	Upper_Estimate	Group
2	1/31/2022	1085431.39	922233.8036	1208628.778	Men's Apparel
3	2/28/2022	1014994	881794.5156	1161191.486	Men's Apparel
4	3/31/2022	1025270.91	882073.4216	1168468.398	Men's Apparel
5	4/30/2022	1018735.07	895537.5816	1181912.558	Men's Apparel
6	5/31/2022	1179009.18	1059811.402	1322306.468	Men's Apparel
7	6/30/2022	1201627.07	1058429.582	1344824.558	Men's Apparel
8	7/31/2022	1248065.22	1103450.347	1393680.093	Men's Athletic Footwear
9	7/28/2022	1245718.01	1001101.137	1290332.883	Men's Athletic Footwear
10	3/31/2022	1208185.59	1063570.717	1352800.463	Men's Athletic Footwear
11	4/30/2022	1278118	1120503.127	1418732.873	Men's Athletic Footwear
12	5/31/2022	1351242.42	1207627.947	1496857.693	Men's Athletic Footwear
13	6/30/2022	1350006.39	1205391.517	1494621.283	Men's Athletic Footwear
14	7/31/2022	1702368.37	1326903.16	2077610.58	Men's Street Footwear
15	2/28/2022	1987204.71	1311835.52	2062569.94	Men's Street Footwear
16	3/31/2022	1672672.21	1297307	2048017.43	Men's Street Footwear
17	4/30/2022	1787566.85	1592171.64	2142902.06	Men's Street Footwear
18	5/31/2022	2082599.84	1707234.45	2457865.07	Men's Street Footwear
19	6/30/2022	2131654.29	1757289.08	2508019.5	Men's Street Footwear
20	1/31/2022	1009487.18	1202521.723	1829450.657	Women's Apparel
21	2/28/2022	1576609.02	1260975.553	1892902.487	Women's Apparel
22	3/31/2022	1614538.51	1298375.083	1930302.017	Women's Apparel

2. Technical Implementation

- Data Preparation (Consistent with Prophet):

```
# --- DATA PREPARATION (WATCHING PROPHET) ---
def prepare_forecast_data(df, metric, groupby=None):
    """Prepare data consistently with Prophet model"""
    df = df.copy()
    df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])

    # Select the appropriate metric column
    metric_col = {
        "Total Sales": "Totalsales",
        "Units Sold": "UnitsSold",
        "Operating Profit": "OperatingProfit",
        "Operating Margin": "OperatingMargin"
    }[metric]

    if groupby:
        # Group by date and the specified column (monthly aggregation)
        forecast_df = df.groupby([pd.Grouper(key='InvoiceDate', freq='M'), groupby])[metric_col].sum().reset_index()
        forecast_df = forecast_df.rename(columns={"InvoiceDate": "ds", metric_col: "y", groupby: "group"})
    else:
```

- Random Forest Forecasting Engine:

```
# --- RANDOM FOREST IMPLEMENTATION ---
def run_rf_forecast(data, periods, n_estimators=100, max_depth=10, groupby=None):
    """Random Forest forecasting with consistent evaluation"""
    if groupby:
        # For grouped forecasts
        groups = data['group'].unique()
        forecasts = {}
        models = {}
        test_data_grouped = {}

        for group in groups:
            group_data = data[data['group'] == group][['ds', 'y']].copy()

            # Convert dates to numerical features (days since start)
            start_date = group_data['ds'].min()
            group_data['days_since_start'] = (group_data['ds'] - start_date).dt.days

            # Add more time-based features
            group_data['year'] = group_data['ds'].dt.year
            group_data['month'] = group_data['ds'].dt.month
            group_data['day'] = group_data['ds'].dt.day
            group_data['dayofweek'] = group_data['ds'].dt.dayofweek
            # Use isocalendar().week and convert to int
            group_data['weekofyear'] = group_data['ds'].dt.isocalendar().week.astype(int)
```

3. Model Evaluation Features

- Consistent Metrics with Prophet:
 - MAE, RMSE, MAPE calculated identically for direct comparison
 - Same visualization of actual vs predicted values
- Additional Features:

Feature Importance Visualization:

- Shows which time components most influence predictions

- Available for each product/region in grouped views

More Flexible Parameter Tuning:

- Users can adjust model complexity
- Immediate feedback on forecast changes

4. Business Value Features

- Comparison Tools:
 - Same interface as Prophet for easy model-to-model comparison
 - Consistent metric calculations for fair evaluation
- Technical Advantages:
 - Handles Non-Linear Relationships: Better for complex patterns
 - Feature Importance: Provides business insights into temporal patterns
 - Configurability: More tuning options for power users

Implementation Details

- Time feature Engineering:

```
# Convert dates to numerical features
start_date = data['ds'].min()
data['days_since_start'] = (data['ds'] - start_date).dt.days

# Add more time-based features
data['year'] = data['ds'].dt.year
data['month'] = data['ds'].dt.month
data['day'] = data['ds'].dt.day
data['dayofweek'] = data['ds'].dt.dayofweek
# Use isocalendar().week and convert to int
data['weekofyear'] = data['ds'].dt.isocalendar().week.astype(int)
```

- Confidence Interval calculation

```
# Calculate confidence intervals
y_train_pred = model.predict(X_train)
residuals = y_train - y_train_pred
std = np.std(residuals)

forecast = pd.DataFrame({
    'ds': future_dates,
    'yhat': y_future,
    'yhat_lower': y_future - 1.96 * std,
    'yhat_upper': y_future + 1.96 * std
})
```

- Evaluation Methodology:
 - Strict 80/20 temporal split (no random splitting)
 - Metrics calculated on most recent 20% of data
 - Same visual presentation as Prophet for consistency
- Business Applications
 - Alternative Forecasting: Provides second opinion to Prophet forecasts
 - Pattern Discovery: Feature importance reveals key time patterns
 - Model Comparison: Helps determine best approach for each metric
 - Parameter Experimentation: Allows tuning for specific use cases

This implementation provides a sophisticated machine learning alternative to Prophet while maintaining the same user experience and evaluation framework, enabling direct comparison between the two approaches.

Key Differences from Prophet:

- Feature Engineering:
 - Converts dates to multiple numerical features:
 - Days since start
 - Year, month, day components
 - Day of week
 - Week of year
- Model Training:

- Uses standard 80/20 train-test split
 - Configurable tree parameters from UI
 - Built-in feature importance calculation
- Confidence Intervals:
 - Calculated from training residuals rather than Prophet's Bayesian approach

Key components Comparison

1. Data Preparation (Common to both)

- Same input data handling
- Identical aggregation methods
- Consistent date formatting
- Matching metric calculations

2. Model Implementation Differences **Prophet**

Model:

```
# --- FORECASTING ---
def run_prophet_forecast(data, periods, confidence_level, groupbyname):
    """Run Prophet forecast on the given data"""
    if groupby:
        # For grouped forecasts
        groups = data.groupby(groupbyname).groups
        forecasts = []

        for group in groups:
            group_data = data[data[groupbyname] == group][["ds", "y"]]

            # handle zero values for multiplicative seasonality
            if group_data["y"].min() <= 0:
                # Additive seasonality for metrics that can be zero or negative
                m = Prophet(seasonality_mode='additive', interval_width=confidence_level/100)
            else:
                # Multiplicative seasonality for strictly positive metrics
                m = Prophet(seasonality_mode='multiplicative', interval_width=confidence_level/100)

            m.fit(group_data)
            future = m.make_future_dataframe(periods=periods, freq='n')
            forecast = m.predict(future)
```

- Advantages:
 - Designed specifically for time series
 - Automatic seasonality detection
 - Handles missing data well
 - Provides uncertainty intervals

Random Forest Model

```
# --- RANDOM FOREST IMPLEMENTATION ---
def run_rf_forecast(data, periods, n_estimators=100, max_depth=10, groupby=None):
    """Random Forest forecasting with consistent evaluation"""
    if groupby:
        # For grouped forecasts
        groups = data['group'].unique()
        forecasts = {}
        models = {}
        test_data_grouped = {}

        for group in groups:
            group_data = data[data['group'] == group][['ds', 'y']].copy()

            # Convert dates to numerical features (days since start)
            start_date = group_data['ds'].min()
            group_data['days_since_start'] = (group_data['ds'] - start_date).dt.days
```

- Advantages:
 - Handles non-linear relationships
 - Configurable model complexity
 - Provides feature importance
 - More flexible for incorporating additional features

3. Visualization Consistency

Both implementations use identical Plotly visualization code for:

- Historical vs forecast plots
- Confidence interval shading
- Tabbed interfaces for grouped forecasts
- Evaluation metric displays

4. Evaluation Methodology

Both models use the same evaluation framework:

- Metrics Calculated:
 - MAE (Mean Absolute Error)
 - RMSE (Root Mean Squared Error)
 - MAPE (Mean Absolute Percentage Error)
- Evaluation Process:

- Temporal train/test split (no random splitting)
- Metrics calculated on held-out recent data
- Identical visual comparison of actual vs predicted

5. Unique Features

Prophet Page:

- Automatic seasonality mode detection (additive/multiplicative)
- Built-in confidence intervals
- Simpler interface with fewer parameters

Random Forest Page:

- Configurable model parameters:
- Number of trees
- Max depth
- Feature importance visualization
- More flexible feature engineering

6. Business Value

- Complementary Approaches:

Prophet: Better for pure time series with strong seasonality Random

Forest: Better for complex, non-linear relationships

- Decision Support:
 - Compare forecasts from both models
 - Understand different perspectives
 - Combine insights for better decisions
- Implementation Benefits:
 - Consistent interface reduces learning curve

- Standardized metrics enable fair comparison
- Modular design allows adding more models

7. Technical Implementation Notes

- Data Flow:

Raw transactions → Prepared time series → Model training → Forecasts → Visualization

- Performance Considerations:
 - Monthly aggregation balances detail and performance
 - Caching of prepared data
 - Efficient grouping operations
- User Experience:
 - Consistent layout between pages
 - Matching interactive elements
 - Similar control panels

Web app meeting the functional requirements

- Admin registration and user authentication

To limit access to authorised users, a login page was made. New admin users can register using the registration system, which securely stores credentials (such as hashed passwords created with the bcrypt library).

After logging in successfully, administrators are taken to the dashboard.

The authentication mechanism guards against unwanted access and guarantees session management.

- Interactive Visualization of Data

The dashboard uses a range of chart types to show important business KPIs and trends:

- Line graphs showing trends in monthly and quarterly sales
- Bar charts to compare the performance of different product categories

- Pie charts showing the regional distribution of sales
- Heatmaps for displaying performance over time and across geographies

With real-time changes to all visual components, interactive widgets let users filter data by time period, product category, or geography.

- Filtering

Datasets can be filtered by users using:

- Year
- Month
- Category of product
- Region

Filtered views can be used to create custom reports, which can then be exported in PDF or Excel format.

- Analysis of Predictive Sales

Sales forecasting for a maximum of a year is implemented. The Prophet model which was trained on previous sales data, is used in this feature.

For the upcoming year, the model forecasts both overall and category-level sales.

A variety of line plots and text summaries are used to display the findings.

Additionally, model accuracy measures (such RMSE) or confidence intervals might be shown.

- Options for Exporting

To assist with reporting requirements:

Datasets and visualizations are available for users to download in PDF, Excel, and CSV formats.

With the help of Streamlit's download widget (`st.download_button`), export buttons were added straight to the dashboard.

- Customization of the Dashboard

Users can customize the dashboard by doing the following:

- Individual KPI visibility can be toggled.
- Select which tables or visualizations to show.
- If persistent storage is enabled, save session preferences.
- Streamlit's layout and widget APIs, along with conditional rendering, were used to accomplish this.
- Importing and Integrating Data

Sales data can be imported into the application from outside sources like:

The Kaggle sales dataset for Adidas

Pandas was used to perform preprocessing on the imported data, which includes feature formatting, data cleaning, and handling of missing values.

Data science solution to Adidas (Data analysis report)

Looking at the dashboard web app, the following analysis can be identified:

Total profit made by Adidas in US in the 2 years: \$47 million

Total sales made by Adidas in US in the 2 years: \$120 million

Total quantity sold by Adidas in US in the 2 years: 2 million

- Total sales by region:

West \$36, 436,157 (30,32%)

Northeast \$25,078,267 (20.87%)

Southeast \$21,374,436 (17.79%)

South \$20,603,356 (17.15%)

Midwest \$16,674, 434 (13,88%)

- Total Profit by region

West \$13,017,584 (27.57%)

Northeast \$9,732,773 (20.61%)

Southeast \$8,393,059 (17.77%)

South \$9,221,605 (19.53%)

Midwest \$6,859,945 (14.53%)

- Total sales by retailer

West Gear \$32,409,588 (26.97%)

Amazon \$10,096,987 (8.4%)

Walmart \$10,506,085 (8.74%)

Kohl's \$13,512,453 (11.24%)

Sports Direct \$24,616,602 (20.49%)

Foot Locker \$29,024,945 (24.15%)

- Total sales by product

Women's Athletic Footwear \$14,315,521 (11.91%)

Women's apparel \$23,870,985 (19.86%)

Women's Street Footwear \$17,201,563 (14.31%)

Men's Athletic Footwear \$20,577,180 (17.12%)

Men's apparel \$16,520,632 (13.75%)

Men's Street Footwear \$27,680,763 (23.04%)

- Total Profit by product

Women's Athletic Footwear \$5,597,822 (11.85)

Women's apparel \$9,686,220 (20.51)

Women's Street Footwear \$6,494,016 (13.75)

Men's Athletic Footwear \$7,437,456 (15.75)

Men's apparel \$6,381,405 (13.51) Men's

Street Footwear \$11,629,045 (24.63)

- Total sales:

Online \$44,965,657

Outlet \$39,536,618

In-Store \$35,664,375

The dashboard web app helps Adidas to understand the sales data over time and predict the future sales based on the historical data.

The design of the dashboard web app for Adidas was based on the data science lifecycle which will go through all the required steps and processors to provide appropriate solutions for the problem.

1. Capture

- This stage involves data acquisition, data entry and data extraction
- This stage involves gathering the relevant dataset, raw structured and unstructured data.
- The sales dataset of Adidas in 2020 and 2021 was obtained from Kaggle, for the analysis of the sales.
- The data is collected from many sources in order to acquire a thorough understanding of the elements influencing sales, and the best and most appropriate source was recognized to be used in order to analyze the data, which would reveal the problem and provide a solution.
- The below diagram represents the Adidas sales dataset obtained.

Retailer	Retailer ID	Invoice Date	Region	State	City	Product	Price per Unit	Units Sold	Total Sales	Operating Prof.	Operating Margin	Sales Method
Foot Locker	1185732	1/1/2020	Northeast	New York	New York	Men's Street Fo	\$30.00	1,200	60,000	30,000	50%	In-store
Foot Locker	1185732	1/2/2020	Northeast	New York	New York	Men's Athletic F	\$30.00	1,000	30,000	15,000	50%	In-store
Foot Locker	1185732	1/3/2020	Northeast	New York	New York	Women's Street	\$40.00	1,000	40,000	14,000	35%	In-store
Foot Locker	1185732	1/4/2020	Northeast	New York	New York	Women's Athle	\$45.00	850	38,250	13,388	35%	In-store
Foot Locker	1185732	1/5/2020	Northeast	New York	New York	Men's Apparel	\$60.00	900	54,000	16,200	30%	In-store
Foot Locker	1185732	1/6/2020	Northeast	New York	New York	Women's Appa	\$30.00	1,000	30,000	12,500	25%	In-store
Foot Locker	1185732	1/7/2020	Northeast	New York	New York	Men's Street Fo	\$30.00	1,250	62,500	31,250	50%	In-store
Foot Locker	1185732	1/8/2020	Northeast	New York	New York	Men's Athletic F	\$30.00	900	45,000	13,500	30%	Outlet
Foot Locker	1185732	1/21/2020	Northeast	New York	New York	Women's Street	\$40.00	950	38,000	13,300	35%	Outlet
Foot Locker	1185732	1/22/2020	Northeast	New York	New York	Women's Athle	\$45.00	825	37,125	12,994	35%	Outlet
Foot Locker	1185732	1/23/2020	Northeast	New York	New York	Men's Apparel	\$60.00	900	54,000	16,200	30%	Outlet
Foot Locker	1185732	1/24/2020	Northeast	New York	New York	Women's Appa	\$30.00	1,000	30,000	12,500	25%	Outlet
Foot Locker	1185732	1/25/2020	Northeast	New York	New York	Men's Street Fo	\$30.00	1,220	61,000	30,500	50%	Outlet
Foot Locker	1185732	1/26/2020	Northeast	New York	New York	Men's Athletic F	\$30.00	925	46,250	13,875	30%	Outlet
Foot Locker	1185732	1/27/2020	Northeast	New York	New York	Women's Street	\$40.00	950	38,000	13,300	35%	Outlet
Foot Locker	1185732	1/28/2020	Northeast	New York	New York	Women's Athle	\$45.00	800	36,000	12,600	35%	Outlet
Foot Locker	1185732	1/29/2020	Northeast	New York	New York	Men's Apparel	\$60.00	850	51,000	15,300	30%	Outlet
Foot Locker	1185732	1/30/2020	Northeast	New York	New York	Women's Appa	\$30.00	950	47,500	11,875	25%	Outlet
Foot Locker	1185732	1/31/2020	Northeast	New York	New York	Men's Street Fo	\$30.00	1,200	60,000	30,000	50%	Outlet
Foot Locker	1185732	2/1/2020	Northeast	New York	New York	Men's Athletic F	\$30.00	900	45,000	13,500	30%	Outlet
Foot Locker	1185732	2/2/2020	Northeast	New York	New York	Women's Street	\$40.00	900	36,000	12,600	35%	Outlet
Foot Locker	1185732	2/3/2020	Northeast	New York	New York	Women's Athle	\$45.00	825	37,125	12,994	35%	Outlet

By accurately gathering important data sources, Adidas can build a structure for thorough analysis and decision-making to solve real-world sales performance issues.

2. Maintain

- During this stage, the collected or obtained data was cleaned and processed before being placed in a format that may be easily used and accessed.

- Continuous data collection and maintenance processes were designed to guarantee that the data available for analysis is up to date.
- Data quality checks and validation methods were implemented to ensure the data's integrity and consistency across time.
- The data sources and available data were continually updated to represent the most suitable and accurate data for analysis.

3. Process

- During the process phase, the goal was to transform and prepare the captured data for analysis.
- To make analysis quicker, the gathered data was organized into a structured manner, including tabular data with rows and columns.
- The data was processed and cleaned to manage missing values, eliminate duplicates, standardize formats, and encode category variables.

4. Analyze

- During the analysis step, the processed data was analyzed, evaluated, and interpreted in order to discover relevant insights and patterns.
- The dataset was integrated into the web app and developed the dashboard web app for Adidas.
- Through the help of the information from the web app, I was able to identify the certain problems within the sales and market of Adidas.

5. Communicate

- In this stage, it involves presenting the findings as visualizations, charts and reports to help stakeholders comprehend the findings.
- Further, the final stage involves translating analytical results into valuable information for Adidas management to make decisions.

Implementation of the solution for Adidas

Utilizing the Dashboard web app for Adidas, the next step was to implement or deploy the data science solution gained from the web app and data analysis.

The data science deployment includes 5 steps:

- Problem definition
- Designing and deployment of the solution
- Monitoring the solution
- Maintaining the solution
- Learn from the solution

1. Problem definition

The first step in implementing a data science solution is to define the problem. This involves clearly articulating the problem statement and objectives.

In this case of Adidas, the objective is to investigate the causes that contributed to the fall in sales from January to March 2021 and to identify methods to avoid future revenue losses.

When considering the year of 2020, at the beginning of the Year it seems that Adidas was able to keep a steady increment in the sales and have been able to increase their sales revenue significantly in 2021. However, there is a decline in the sales revenue and profits in the first 3 months of the year 2021.

The below output from the web app represents the decline in sales in the first 3 months of 2021.



- The sales revenue has been decreased from \$7Million in January to \$6 Million in February to \$5 Million in March.
- The obtained data was used in order to find the problem that cause this lost to Adidas.
- It was identified that this reduction has been linked to the problem which is the Covid-19 pandemic.
- The pandemic caused interruptions in the worldwide supply chain resulting in store closes and limiting Adidas' ability to manufacture and deliver its product items.
- This significantly impacted Adidas's sales, as many consumers were staying home and prioritizing essential purchases over discretionary items such as athletic apparel and footwear.
- COVID-19 has certainly presented an array of problems for Adidas, in addition to the drop in revenue. These problems include:

○ Disruptions in supply chain

The pandemic has affected global supply chains, resulting from manufacturing facility closures, transit delays, and raw material shortages. This disruption made an impact on Adidas' capacity to manufacture and deliver products to clients on time, resulting in inventory concerns and significant loss of sales.

- Reduction in consumer spending

Economic uncertainties and job losses caused by the pandemic influence consumer sentiment and buying patterns. Consumers have become more price conscious and prioritize critical purchases over expensive products such as sportswear and apparel.

Adidas's sales in the US and also worldwide were impacted by high unemployment rates, less discretionary income, and nervousness about the future, leading to a more cautious approach to spending by consumers.

- Shift to online shopping

With physical retail sites limited and not accessible, there has been a huge shift to online shopping. While Adidas has an online presence, the increased volume of online orders has put a strain on e-commerce sites, logistics, and fulfillment processes.

Furthermore, greater competition in the online space has necessitate further investments in digital marketing and client interaction techniques.

2. Designing and deployment of the solution

In this stage, all the data were collected from data sources, providing insights into Adidas's sales performance, market dynamics, consumer habits and external influences such as Covid - 19.

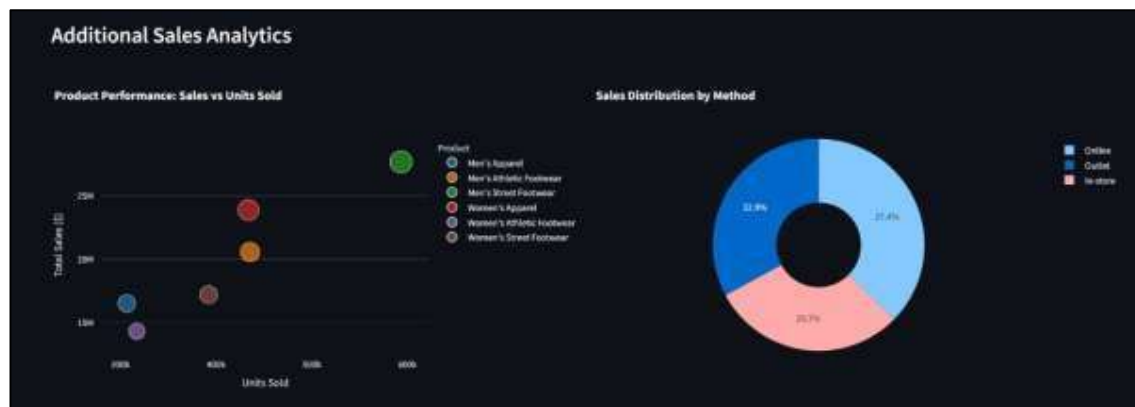
This data collection contained internal sales data based on the region, state and time, as well as the sales method and the specific retailer assigned to the region.

When structured, this included data on total sales revenue, units sold, and profit for each month.

The data was well structured and it was loaded into a data-free processing and visualization platform that produced a dashboard with all of the critical details that will be used to obtain a data science solution to the problem of declining sales.

Using the dashboard web app that analyzed information, a data science solution will be developed to increase the sales of Adidas in the upcoming months of 2021.

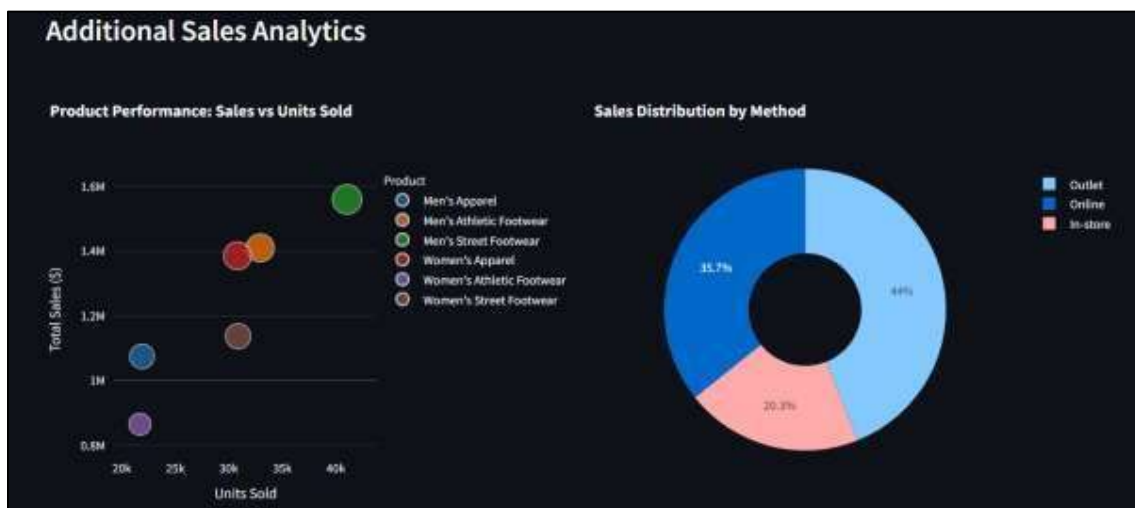
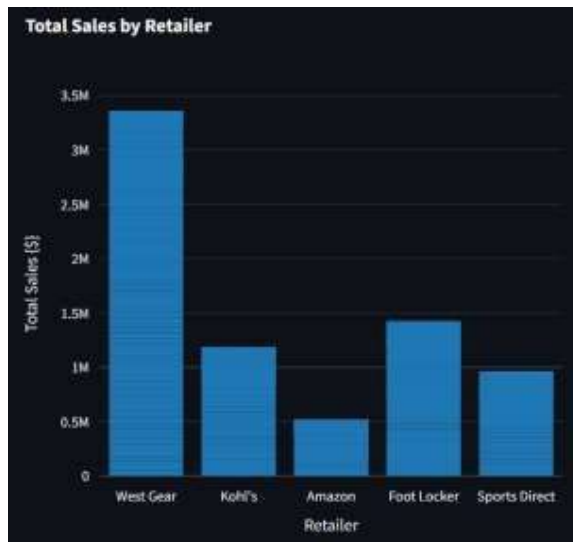
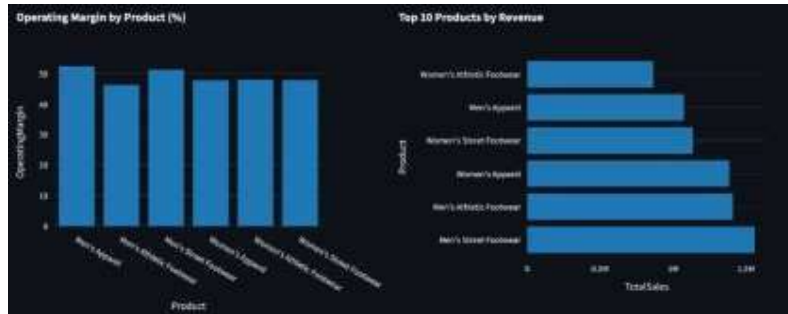
The below diagram represents a part of the dashboard web app.



The below diagrams represent the comparison of sales data in those 3 months:

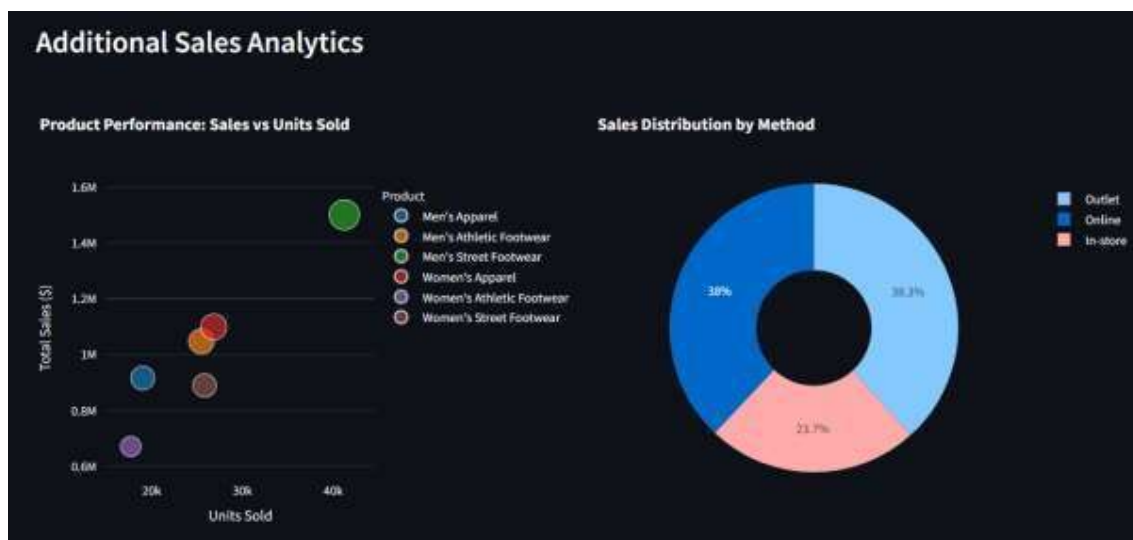
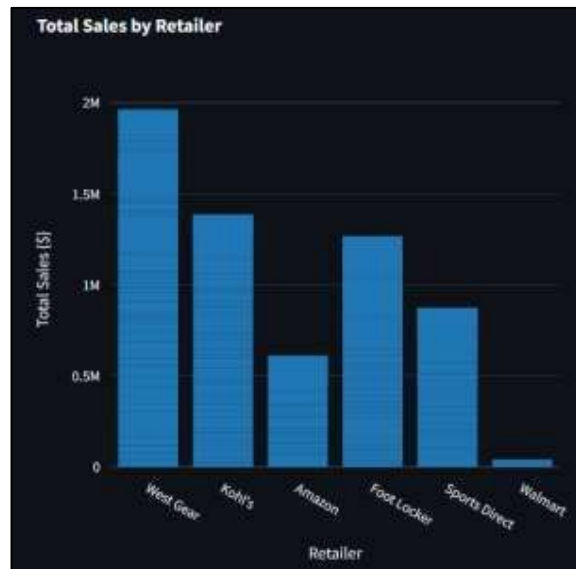
○ January 2021





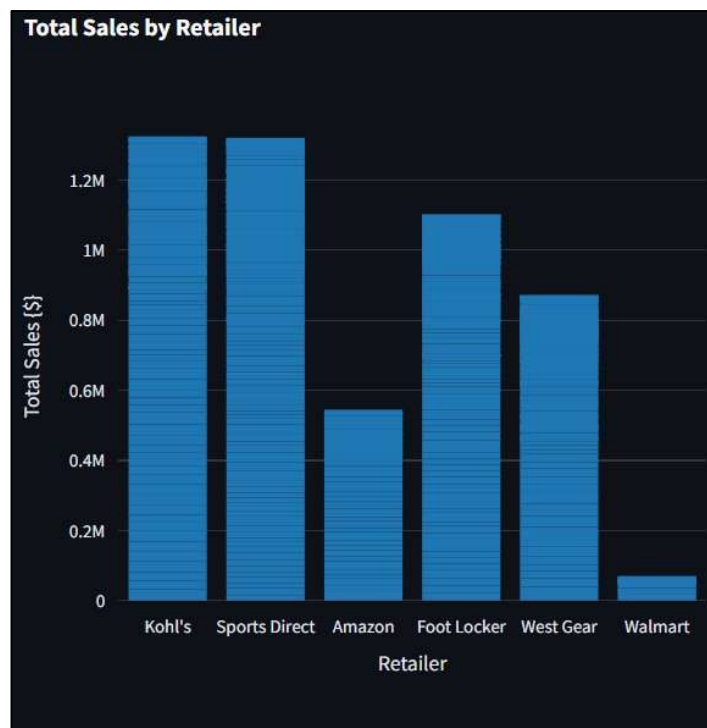
○ February 2021

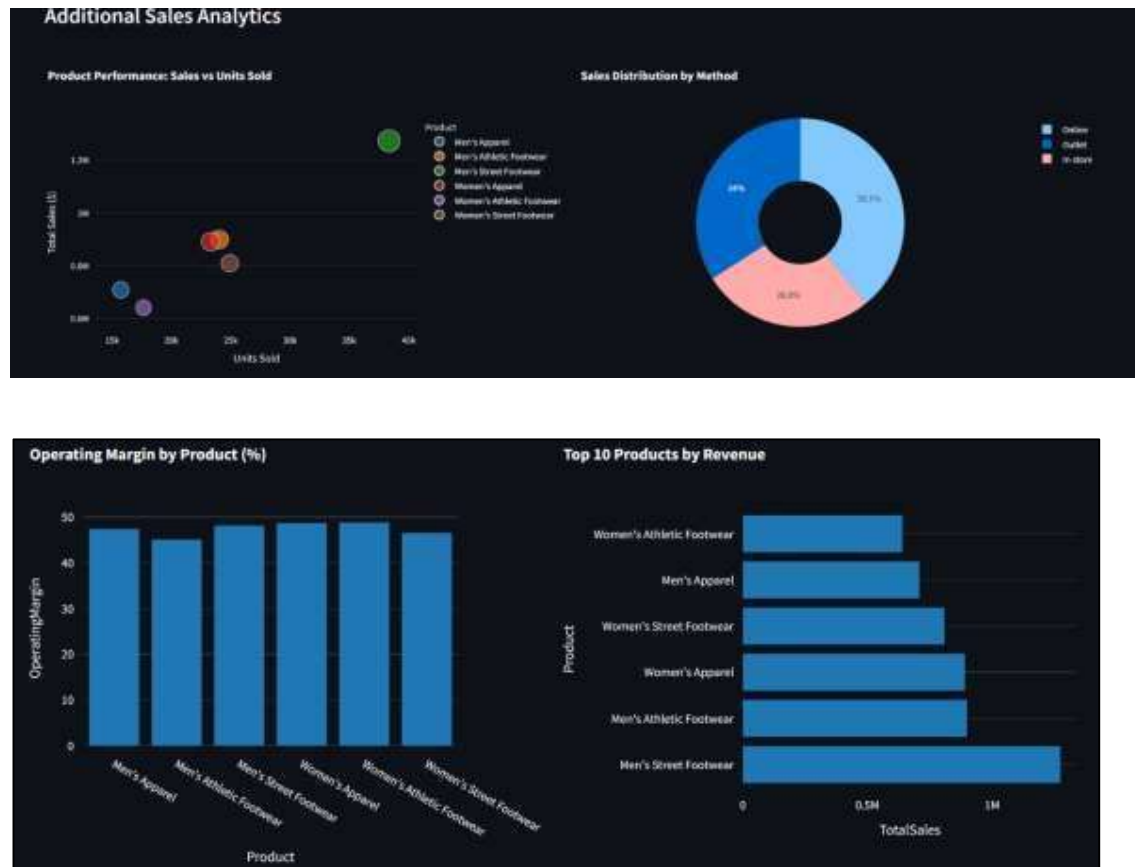
Select Year	Total Sales	Total Quantity Sold	Total Profit
2021	\$6,123,040.00	156,113	\$2,282,519.76
Select Month			
February			





○ March 2021





Solutions

With the above Analytics, a conclusion can be made that there has been a significant decrease in the sales within these three months of January, February and March.

When considering the three main problems which was developed with the COVID-19 Pandemic, solutions have to be provided in order to overcome the problems which will ensure that the sales in the upcoming months of 2021 will be increased and maintained at a steady level.

The solutions to be deployed will be as follows:

I. Problem: Disruptions in supply chain

○ Solution:

Adidas should undertake methods to improve supply chain resilience and diversify its sources of supply to reduce the effect of disruptions.

Establishing connections with suppliers is also critical for supply chain resilience and efficient administration.

Having numerous manufacturing locations in different regions or states helps Adidas to reduce the likelihood of production disruptions.

○ Deployment steps:

- Diversify sourcing locations

Adidas have to discover alternate manufacturing regions to avoid reliance on a single sourcing site which is susceptible to disruptions.

Further, they have to form partnerships with numerous vendors for crucial components such as raw materials and supplies.

- Enhance supplier relationships

Adidas have to increase their relationships with suppliers to promote transparency and communication and conduct regular performance evaluations and quality reviews to ensure dependability.

- Developing contingency plans

Adidas will have to develop backup plans for several supply chain situations, such as transportation difficulties, shortages of materials, or delays in manufacturing.

To reduce further risks, Adidas will have to consider alternate sourcing and logistical suppliers as well.

II. Problem: Reduction in consumer spending

○ Solution:

Developing innovative products to meet evolving consumer demands and engage with customers through targeted marketing.

Adidas can offer customization possibilities for footwear and gear where customers can customize their apparel.

Adidas can increase brand loyalty and engagement by allowing customers to design personalized products.

- Deployment steps:

- Product innovation

Adidas have to invest in new product development to provide unique and appealing products that will continue to stand out in the competitive marketplace at a price which is affordable

Further, to appeal to customers, prioritize sustainability, performance, and style.

- Partnerships and collaborations

Form strategic alliances with other companies, designers, or influencers to develop unique collections or limited-edition products that will create interest and boost sales.

Engage fans in the process of design using contests, polls, or immersive experiences to create a sense of community and involvement.

- Customer experience improvements

Adidas can enhance customer service and afterwards to promote loyalty and repeat purchases and also offer outstanding support, quick shipping, and easy returns.

- Promoting health and wellness initiatives

Adidas's marketing strategies align with health and wellness trends, positioning it as a lifestyle brand that encourages physical activity and well-being.

Adidas can develop content highlighting Adidas products' health and performance advantages, including workout suggestions, training schedules, and athlete stories.

III. Problem: Shift to online shopping

○ Solution:

Adidas should undertake approaches to react to the change to online shopping, such as increasing its digital presence and concentrating on multichannel marketing.

This includes enhancing the online buying experience, utilizing social commerce platforms, and improving digital marketing initiatives.

Adidas can close the gap across its digital and physical outlets by employing omnichannel strategy. This includes click-and-collect services which allow customers to buy things online and pick them up at nearby Adidas shops, reducing the distinction among online and offline shopping.

○ Deployment steps:

- Improve online shopping experience

Adidas will have to improve both the website and mobile app to deliver a more smooth and user-friendly experience for customers and make product search capabilities, product pages, and the checkout procedure simpler.

- Increase social media platforms

Adidas will have to use more social media platforms including twitter, Facebook and Instagram to promote shoppable content, live streaming events, and celebrity partnerships.

This will allow direct purchases through social media platforms to leverage on consumer interaction and impulsive buying.

- Invest in e-commerce infrastructure

Adidas will need to enhance its online purchasing platform to accommodate greater traffic and create a seamless experience. Upgrading the user interface and website content will keep customers loyal to Adidas and enhance engagement.

- Optimize digital marketing campaigns

Adidas will have to establish personalized digital marketing initiatives that are tailored to the tastes and habits of the online audience.

Adidas can implement email marketing, and social media marketing to contact potential buyers through various channels.

- Establishing omnichannel approaches

Incorporate both digital and physical channels to give customers a smooth purchasing experience. This will address the gap among digital and physical retail by providing clickand-collect services

To create a single brand personality and consumer experience, Adidas will have to ensure that branding, prices, and advertising are consistent across all channels.

3. Monitoring the solution

- After implementing solutions to address the identified problems Adidas must continually monitor the effectiveness of these efforts and make required improvements. This includes tracking key performance indicators (KPIs) and assessing the effectiveness of marketing campaigns.
- To check whether the solution is effective, the sales revenue of the upcoming months of 2021 after March can be monitored to assess whether there is an improvement in the sales.
- The previous diagrams represent that there is significant decrease in the sales in first 3 months of 2021 even though the sales revenue is higher in 2021 compared to 2020.
- The data science solution designed and implemented is expected to enhance sales by 1 million or more by the next month (April).
- These data science solutions will reduce the existing problem and provide other techniques to increase the sales value and will result in an increase in profit as well.
- By monitoring revenue overall patterns and the sales graph, Adidas will be able to evaluate the effect of deployed solutions on sales performance and determine whether the recommended, designed, and deployed data science solution was successful.
- The below diagram represents the sales information in the next months (April, May, June) after the implementation of the solution.



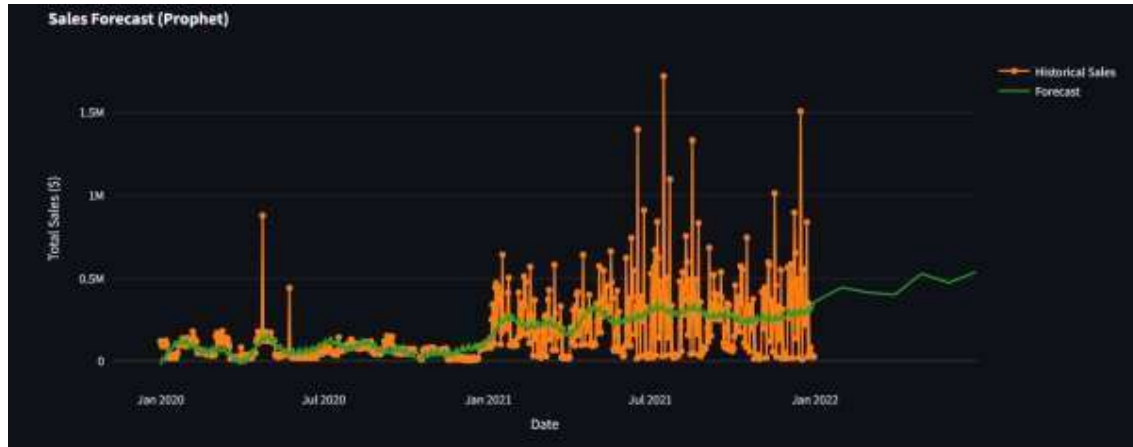
- The above diagrams represent that the sales of Adidas have increased compared to the first 3 months of 2021.
- This claims that the data science solution implemented has been successful.
- The client's objective of increasing sales by \$1 million or more was met, leaving the client satisfied that we had achieved our ultimate goal of offering the finest data science solution to the existing problem.
- Increased sales also suggest that customers buy more products, which leads to the conclusion that this data science solution has met the customer's expectations.

4. Maintaining the solution

- When the solutions of increasing sales of Adidas have been implemented, maintaining the solution involves continuous monitoring and improvements
- Using the dashboard web app, the sales data for the next year can be analyzed to determine whether the data science solution has been maintained through the result of increase in sales revenue.
- The continuous maintenance on sales strategies based on insights gain from this data will improve the profits of Adidas as well.
- The sales gained in the next 5 months of the year 2021 after March have been increased as expected due to the data science solution.



- The below diagrams from the dashboard web app represent the sales information of the year 2022 (Predictive analysis).



Forecast Data Table

	Date	Predicted Sales	Lower Estimate	Upper Estimate
0	Jan 2022	\$443,714.25	\$236,409.48	\$664,116.21
1	Feb 2022	\$414,377.88	\$194,058.09	\$618,697.58
2	Mar 2022	\$399,523.77	\$181,642.15	\$605,482.80
3	Apr 2022	\$526,518.78	\$300,231.65	\$732,208.16
4	May 2022	\$471,187.14	\$239,409.57	\$692,777.29
5	Jun 2022	\$538,334.14	\$315,852.02	\$756,705.63

5. Learn from the solution

This is the final phase of the data science solution deployment.

This Data Science Solution, which was used to increase Adidas's sales for a few months by measuring sales indicators, will be useful for Adidas in the future as well.

With this, Adidas will be able to evaluate total sales performance over time and analyze sales trends, resulting in customer happiness and an increase in profits for the company.

Finally, the continuous tracking of sales metrics through data dashboards and visualization from the dashboard web app can discover patterns, trends and areas for improvement in the future.

Overall, Adidas can promote continuous improvement, retain its competitive advantage, and achieve long-term success in the dynamic retail market environment by methodically learning from deployed solutions and continuously improving plans based on insights obtained.

Justification of the recommendations that support decision making in reference to the real-world problem

A comprehensive justification will be made for the recommendations and solutions provided by the data science solution model in order to explain why these specific data science solutions were chosen.

- Supply chain resilience

Justification:

The COVID-19 epidemic has disrupted the worldwide supply chain, highlighting the requirement for Adidas to strengthen its supply chain resilience. Adidas can decrease risks, increase responsiveness, and maintain continuous supply chain operations by diversifying sourcing locations, building supplier relationships, deploying inventory management technologies, and establishing contingency plans.

Decision support:

These recommendations are supported by the awareness that supply chain interruptions are a key contributor to sales declines. By improving supply chain resilience, Adidas is able to adjust to unexpected difficulties and preserve business continuity, allowing for more informed decision-making to reduce future risks.

Tracking metrics relating to supply chain performance, including inventory levels, lead times, supplier performance, and manufacturing capacity, can help Adidas understand the wellbeing and effectiveness of its supply the network.

- Enhance digital shopping experience

Justification:

Adidas must improve its digital purchasing experience to satisfy changing consumer attitudes and behaviors, since this pandemic has accelerated the switch to online shopping. Adidas can deliver a seamless and interactive shopping experience and improve customer loyalty by

investing in e-commerce infrastructure, adding virtual try-on tools, improving mobile app capabilities, and using social commerce.

Decision support:

The observed decline in foot traffic in physical stores, as well as the increasing significance of online channels in increasing sales, support the proposal to improve the digital shopping experience. Adidas can efficiently reach and attract customers by concentrating on omnichannel strategies and digital innovation, allowing for data-driven decision-making to optimize sales channels.

Tracking metrics such as online sales, website traffic, conversion rates, and customer happiness can assist Adidas in determining the efficiency of its digital initiatives and identifying areas for improvement.

- Adapting to changing consumer preferences due to changes in consumer spending

Justification:

Changes in consumer behavior, such as a preference for athleisure wear and a focus on health and wellbeing, require Adidas to adapt its product offerings and marketing techniques. Adidas can stay relevant to shifting consumer tastes by creating sustainable and adaptable products, establishing collaborations and limited editions, personalizing marketing messaging, and growing digital community participation.

Further, targeted marketing initiatives help Adidas reach and attract certain customer categories, increasing brand exposure, consideration, and sales.

Decision support:

The recommendation for adapting to changing consumer preferences is supported by an understanding of changing market conditions and the significance of meeting customer expectations. Adidas can maintain its competitiveness and promote sales growth by improving products and marketing tactics, allowing for more informed decision-making in order to capture market opportunities and increase brand loyalty.

Overall, these recommendations are supported by Adidas' recognized problems which include supply chain interruptions, changes in customer behavior, and the transition to online buying.

By tackling these difficulties with tailored methods, Adidas can promote informed decisionmaking and establish themselves for long-term success in the retail market.

CHAPTER 05 VALIDATION

This section covers how I evaluated the performance and the reliability of the predictive models and the test cases of the dashboard web app.

Model Validation Approach

Evaluation Methodology

Both models use the same evaluation framework:

- Metrics Calculated:
 - MAE (Mean Absolute Error)
 - RMSE (Root Mean Squared Error)
 - MAPE (Mean Absolute Percentage Error)
- Evaluation Process:
 - Temporal train/test split (no random splitting)
 - Metrics calculated on held-out recent data
 - Identical visual comparison of actual vs predicted

```
def calculate_metrics(y_true, y_pred, metric_name):  
    """Consistent metric calculation with Prophet"""  
    mae = mean_absolute_error(y_true, y_pred)  
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))  
    mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100  
  
    return {  
        "MAE": mae,  
        "RMSE": rmse,  
        "MAPE": mape  
    }
```

- Key Points on Model Evaluation Periods:

Prophet Model Evaluation: The historical performance metrics (MAE, RMSE, MAPE) for the Prophet model were calculated by training the model on the historical data excluding the last 3 months and then evaluating its predictions against those last 3 months.

Random Forest Model Evaluation: The historical performance metrics for the Random Forest Regressor were calculated by training the model on the historical data excluding the last 20% and then evaluating its predictions against that last 20% of the data.

Implication: Because the test periods used for calculating these evaluation metrics are different (last 3 months vs. last 20% of data), the metrics for the two models are not directly comparable over the exact same historical time window.

For the purpose of evaluating the historical performance of the predictive models, a train-test split approach was utilized. The Prophet model's evaluation metrics (MAE, RMSE, MAPE) were calculated based on forecasting the last 3 months of the historical data.

In contrast, the Random Forest Regressor's evaluation metrics were derived from predicting the last 20% of its respective historical data. It is important to note that due to this difference in the test set time periods, the evaluation metrics provided for the two models are not directly comparable in a strict, like-for-like manner. Each model's metrics reflect its performance over a distinct portion of the historical timeline.

Example:

Model evaluation metrics for overall total sales:

Prophet Model:

Model Evaluation		
Evaluation for Overall Total Sales:		
MAE	RMSE	MAPE
\$1,362,710.68	\$1,541,616.76	17.88%

Random forest Regressor:



To assess the performance of the models in predicting Overall Total Sales, three common evaluation metrics were used: MAE, RMSE, and MAPE. These metrics help determine how close the predicted values are to the actual sales data.

1. Mean Absolute Error (MAE)

MAE measures the average absolute difference between predicted and actual values, without considering the direction of the error.

On average, how much did the model's predictions differ from the actual sales figures (in dollar terms)?

Interpretation: Lower MAE = Better accuracy.

• Results:

- Prophet: \$1,362,710.68
- Random Forest: \$1,279,294.73
- Conclusion: Random Forest had a lower MAE, meaning its predictions were closer to actual values by an average of \$83,415.95 compared to Prophet.

2. Root Mean Squared Error (RMSE)

RMSE is similar to MAE but squares the errors before averaging and then takes the square root. This means larger errors are penalized more heavily.

In simple terms: It shows how significant the prediction errors were, with more weight given to large mistakes.

Interpretation: Lower RMSE = Fewer large errors.

• Results:

- Prophet: \$1,541,616.76
- Random Forest: \$1,506,394.65
- Conclusion: Random Forest again performed better, with a slightly lower RMSE, indicating it handled large errors more effectively.

3. Mean Absolute Percentage Error (MAPE)

MAPE expresses the average error as a percentage of the actual values.

In simple terms: It shows how far off the predictions were, on average, in percentage terms.

Interpretation: Lower MAPE = Better relative accuracy.

- Results:
 - Prophet: 17.88%
 - Random Forest: 16.36%
 - Conclusion: Random Forest achieved a 1.52 percentage point improvement over Prophet, showing more accurate predictions in relative terms.

Test Cases

Test Case	Description	Input/Test steps	Expected Output	Status
TC01	User login with valid credentials	Username + Password	Redirect to dashboard	Pass
TC02	User register to access the system	Username + Password + Email	Registration Successful! Please Login	Pass
TC03	Invalid login attempt	Wrong password	Error message	Pass
TC04	Upload valid Excel sales file	Adidas.xlsx	Data loaded successfully	Pass
TC05	Upload invalid file format	image.jpg	Error message	Pass

TC06	Upload corrupted Excel file	Corrupted.xlsx	Error message	Pass
TC07	Filter by Year	2020/2021	Filtered data	Pass
TC08	Filter by product category	Select "Men's Apparel"	Filtered visualization	Pass
TC09	Filter by month	Select "January"	Month-specific data	Pass
TC10	Filter by Retailer	Select "Walmart"	Walmart specific data	Pass
TC11	Filter by region	Select "West"	West – specific data	Pass
TC12	Reset all filters	Click "Reset Filters"	Default view restored	Fail
TC13	Retailer sales bar chart	Apply filters	Updated chart	Pass
TC14	Time series line chart	Change time aggregation	Smooth line graph	Pass
TC15	State-wise sales dual-axis chart	Hover interaction	Tooltip with values	Pass
TC16	Treemap region/city view	Click on region	Drill-down works	Pass
TC17	Product performance chart	Apply filters	Updated chart	Pass
TC18	Sales distribution pie chart	Apply filters	Updated chart	Pass
TC19	Dashboard header	Load the dashboard	Logo + title displayed correctly	Pass
TC20	KPI metrics formatting	Apply currency filters	"\$12,345.67" format with 2 decimal places	Pass
TC21	Last updated timestamp	Refresh data	Timestamp updates to current time	Pass

TC22	Sidebar filter persistence	Reload page after setting filters	Filters retain state	Pass
TC23	Dark mode compatibility	Toggle dark mode	All charts adapt color scheme	Pass
TC24	KPI metrics formatting	Apply currency filters	"\$12,345.67" format with 2 decimal places	Pass

TC25	Missing data points for charts	Filter to sparse time range	Line breaks with gaps	Pass
TC26	Hover tooltip values	Hover over Q1 2021 point	Shows exact sales: "\$1.2M"	Pass
TC27	Color contrast on charts	Check label readability	White text on dark colors	Pass
TC28	Secondary axis toggle	Click "Hide Units Sold"	Only sales bars remain	Pass
TC29	Cross-chart filtering	Click "Amazon" in retailer chart	All other charts filter to Amazon	Fail
TC30	Brush selection in time series	Drag-select Jan-Mar 2024	All charts zoom to range	Pass
TC31	Legend toggle	Click "Men's Apparel" in legend	Hides category across all charts	Pass
TC32	Reset zoom button	Click after zooming	Returns to default view	
TC33	Cross-chart filtering	Click "Amazon" in retailer chart	All other charts filter to Amazon	Pass

TC34	PNG export of chart	Click "Export as PNG" on treemap	Image matches screen view	Pass
TC35	CSV data matches chart	Export retailer sales CSV	Numbers equal hover tooltip values	Pass
TC36	Print layout	Browser print preview	Charts fit page, no cutoffs	Pass
TC37	Screen reader support	Navigate with NVDA	Announces chart types + summaries	Pass
TC38	Keyboard navigation	Tab through filters + charts	Focus rings visible	Pass

TC39	Colorblind mode	Enable deuteranopia simulation	Charts use patterns + safe palette	Fail
TC40	Move to a different page	Select "Predictive analysis"	Redirects to predictive analysis page	Pass
TC41	Prophet sales forecast	3-month horizon	Future Trend line	Pass
TC42	Prophet sales forecast	6-month horizon	Future Trend line	Pass
TC43	Prophet sales forecast	12-month horizon	Future Trend line	Pass
TC44	Select metric to forecast in predictive analysis page	Select "Units sold"	Units sold forecasts related data	Pass
TC45	Change forecast aggregation level	Select "By product"	Updated data with forecasts by product	Pass

TC46	Confidence Interval Adjustment	Set to 90%	Wider prediction bands	Pass
TC47	Forecast comparison tool	Select to compare “Men’s Apparel and Women’s apparel”	Updated forecast chart related to men’s apparel and women.	Pass
TC48	Model Evaluation change	Select “Total sales” in predictive analysis	The model evaluation updates (MAE, RMSE)	Pass
TC49	Moving to a Advanced predictive analysis page	Select “Advanced analysis”	Redirects to that page	Pass
TC50	Changing metric in random forecast	Select “Operational profit”	Updated charts and data related to	Pass
			operational profits forecasts	
TC51	Change forecast aggregation level in random forest page	Select “By product”	Updated data with forecasts by product	Pass
TC52	Feature importance for random forest model	Selecting a particular filter	The charts get updated	Pass
TC53	Change random forest forecast aggregation level	Select “By product”	Updated data with forecasts by product	Pass
TC54	Model parameters Adjustment	Set to 90%	Wider prediction bands	Pass

TC55	No data matching filters	Impossible data range	Empty charts	Pass
TC56	Download retailer sales CSV	Click download button	CSV file generated	Pass
TC57	Download monthly sales CSV	Click download button	CSV file generated	Pass
TC58	Download raw data CSV	Click download button	CSV file generated	Pass
TC59	Download Prophet forecasts sales CSV	Click download button	CSV file generated	Pass
TC60	Download Random Forest forecast sales CSV	Click download button	CSV file generated	Pass

CHAPTER 6

CONCLUSION & FUTURE WORK

In conclusion, managing the security challenges in big data within organizations is important for organizations to secure sensitive information, maintain regulatory compliance, and retain stakeholder trust. Unauthorized access, data breaches and data volume had the most significant impact on organizations according to the survey findings and interview. To effectively tackle these challenges, organizations must take a comprehensive strategy that includes strategies such as strong access controls, encryption, data governance, and incident response mechanisms.

Key solutions included advanced encryption techniques, access control mechanisms, training for employees, data masking techniques and utilizing extensive security frameworks. By utilizing the strategies and recommendation recommended, organizations can effectively mitigate the security challenges in big data and also establish a secure big data environment within the organization.

In the context of the development of the dashboard web app, several meaningful future work directions include:

1. Enhanced Predictive Modeling

- Implement more advanced machine learning models like XGBoost, LSTM neural networks, or ensemble methods to potentially improve forecasting accuracy
- Develop hybrid models that combine the strengths of statistical approaches (like Prophet) with machine learning techniques
- Incorporate external variables like economic indicators, weather data, or marketing campaigns that may affect sales

2. Real-time Analytics Integration

- Develop a real-time data pipeline to analyze streaming sales data as it comes in
- Implement automated anomaly detection to flag unusual sales patterns immediately
- Create alert systems for when sales deviate significantly from forecasts

3. Expanded Business Intelligence Features

- Add customer segmentation analysis to identify high-value customer groups
- Incorporate inventory optimization recommendations based on sales forecasts •
Develop price optimization models to suggest ideal pricing strategies

4. Improved User Experience

- Implement natural language processing to allow users to query the dashboard using plain English
- Add automated report generation that executives can schedule and receive via email
- Develop mobile-friendly versions of the dashboard for on-the-go access

5. Deployment Enhancements

- Containerize the application using Docker for easier deployment
- Implement user authentication and role-based access controls
- Set up automated testing and continuous integration/continuous deployment (CI/CD) pipelines

6. Data Expansion

- Incorporate additional data sources like social media sentiment, competitor pricing, or supply chain information
- Add more granular geographic data for hyper-local sales analysis
- Include product return data to analyze the full customer lifecycle

7. Advanced Visualization

- Implement augmented reality (AR) visualizations for immersive data exploration
- Add geospatial mapping capabilities for location-based insights
- Develop interactive what-if scenario tools for business planning

REFERENCES

AL-SAI, Z., ABDULLAH, R. & HUSIN, H. (2019) Big Data Impacts and Challenges: A Review. *ResearchGate*. [Online] Available from: https://www.researchgate.net/publication/332543555_Big_Data_Impacts_and_Challenges_A_Review [Accessed: 02/02/2025]

ANALYTIXLABS (2023) Random Forest Regression — How it Helps in Predictive Analytics? *AnalytixLabs*. [Online] Available from: <https://medium.com/@byanalytixlabs/random-forest-regression-how-it-helps-in-predictiveanalytics-01c31897c1d4> [Accessed: 20/05/2025]

AWATI, R. Advanced Encryption Standard (AES) *TechTarget*. [Online] Available from: <https://www.techtarget.com/searchsecurity/definition/Advanced-Encryption-Standard> [Accessed: 14/05/2025]

AYESH, S. (2025) *Interview with S. Ayesha on 27th March 2025*. [Recording in possession of author]

BHATTACHARYA, A. (2024) Big Data - Data Encryption in Big data. *ENCRYPTION CONSULTING*. [Online] Available from:
<https://www.encryptionconsulting.com/dataprotection-in-big-data-using-encryption/>
[Accessed: 14/05/2025]

BENJELLOUN, Z, F & LAHCEN, A, A. (2015) Big Data Security: Challenges, Recommendations, and Solutions. *ResearchGate*. [Online] Available from:
https://www.researchgate.net/publication/278962714_Big_Data_Security_Challenges_Recommendations_and_Solutions [Accessed: 04/02/2025]

BIGELOW, J, S. (2021) 4 Best Practices for Big Data Security in Cloud Computing. *TechTarget*. [Online] Available from:
<https://www.techtarget.com/searchcloudcomputing/tip/4-best-practices-for-big-data-security-in-cloud-computing> [Accessed: 12/02/2025]

COBB, M. (2022) data masking. *TechTarget*. [Online] Available from:
<https://www.techtarget.com/searchsecurity/definition/data-masking> [Accessed: 16/05/2025]

DUTTA, S. (2024) Big Data Security: Advantages, Challenges, and Best Practices. *Turing*. [Online] Available from: <https://www.turing.com/resources/big-data-security#2.-effectiveuser-access-control> [Accessed: 14/05/2024]

ELEVONDATA (2023) The Top Challenges of Big Data: Volume, Velocity, Variety, and Veracity. *Elevondata*. [Online] Available from: <https://www.linkedin.com/pulse/topchallenges-big-data-volume-velocity-variety-veracityelevondata#:~:text=Volume%20is%20one%20of%20the,%2C%20sensors%2C%20and%20transactional%20systems>. [Accessed: 02/05/2025]

GAUR, C. (2023) *Big Data Security Tools and Management Best Practices*. [Online] December 4th 2023. Available from: <https://www.xenonstack.com/blog/big-data-security>
[Accessed: 15th April 2025]

- GEEKSFORGEEKS. (2024) Time Series Analysis Using Facebook Prophet. *GeeksforGeeks*. [Online] Available from: <https://www.geeksforgeeks.org/time-series-analysis-usingfacebook-prophet/> [Accessed: 17/05/2025]
- GEORGE, T. (2021) Mixed Method Research | Definition, Guide & Examples. *Scribbr*. [Online] Available from: <https://www.scribbr.com/methodology/mixed-methods-research/> [Accessed: 21/04/2025]
- GILLIS, A (2021) principle of least privilege (POLP). *TechTarget*. [Online] Available from: <https://www.techtarget.com/searchsecurity/definition/principle-of-least-privilege-POLP> [Accessed: 03/05/2025]
- GILLIS, A. (2024) What is role-based access control (RBAC)? *TechTarget*. [Online] Available from: <https://www.techtarget.com/searchsecurity/definition/role-based-accesscontrol-RBAC> [Accessed: 14/04/2025]
- JANSEN, D. (2023) Research Philosophy & Paradigms. *GRADCOACH*. [Online] Available from: <https://gradcoach.com/research-philosophy/> [Accessed: 10/05/2025]
- JUMBOBLOCKCHAIN. (2023) *Blockchain and Big Data: Enhancing Data Security and Privacy*. [Online] May 12th 2023. Available from: <https://www.linkedin.com/pulse/blockchain-big-data-enhancing-security-privacyjumbochainglobal/> [Accessed: 15th May 2025]
- KUMAR, D. (2021) Top 7 Big Data Security Issues and Their Solutions. *Hevo*. [Online] Available from: <https://hevodata.com/learn/big-data-security/#et> [Accessed: 01/05/2025]
- LAWTON, G. (2022) DataOps. *TechTarget*. [Online] Available from: <https://www.techtarget.com/searchdatamanagement/definition/DataOps> [Accessed: 16/05/2022]
- LUTKEVICH, B. (2023) tokenization. *TechTarget*. [Online] Available from: <https://www.techtarget.com/searchsecurity/definition/tokenization> [Accessed: 14/05/2025]
- MCKINSEY & COMPANY (2016) How companies are using big data and analytics. *MCKINSEY & COMPANY*. [Online] Available from: <https://www.mckinsey.com/capabilities/quantumblack/our-insights/how-companies-areusing-big-data-and-analytics> [Accessed: 14/04/2025]
- POUR, H, C. (2024) big data. *TechTarget*. [Online] Available form:

<https://www.techtarget.com/searchdatamanagement/definition/big-data> [Accessed: 29/04/2025]

SRIRAM, K, G. (2022) Security Challenges of Big Data Computing. *ResearchGate*. [Online] Available from: https://www.researchgate.net/publication/358226925_SECURITY_CHALLENGES_OF_BIG_DATA_COMPUTING [Accessed: 06/04/2025]

TECEZE (2024) Potential Impact of AI: Data Security Challenges. *TECEZE*. [Online] Available from: <https://teceze.com/potential-impact-of-ai-data-securitychallenges#:~:text=The%20Potential%20of%20AI%20in%20Data%20Security,-AI%20holds%20great&text=Machine%20learning%20algorithms%20can%20glean,sensitive%20information%20and%20potential%20vulnerabilities.> [Accessed: 01/05/2025]

APPENDIX

The source code for the Dashboard web app is available on GitHub

Dashboard web app source code: <https://github.com/Senash13/Final-yearproject/tree/main/Dashboard%20web%20app>

This repository contains all the code, templates, and resources required to run the app.

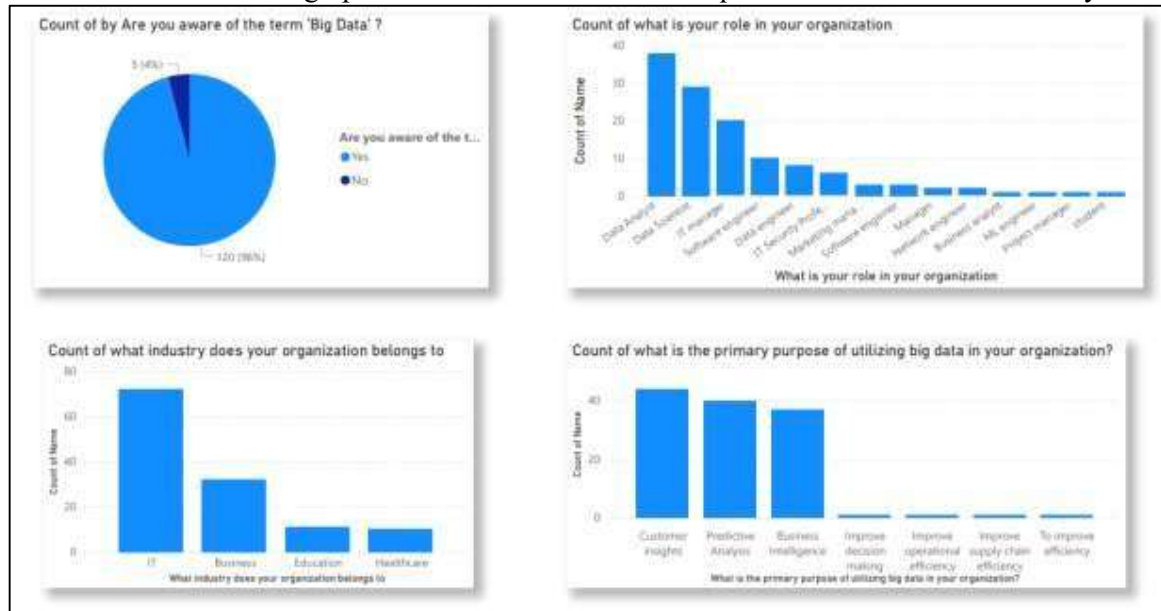
To run the app, clone the repository, create a Python virtual environment, install dependencies via requirements.txt, and run app.py. The app will be accessible on <http://localhost:5000>.

- Google form responses excel sheet:

<https://1drv.ms/x/c/cbf9328fd5aa5a66/EYZG8Jb4YedAg19GvZsKacBCQ94HXIJbcchcqS91Wq7Lw>

Google form graphs

These are the additional graphs which were visualized the questions included in the survey.



Interview transcript

Senash: Good afternoon, sir, thank you for participating for the interview

Mr Ayesh: Hi Senash

Senash: Hi sir so let's start the interview

Mr Ayesh: Ok Ok sure, lets start

Senash: So the first question is are you aware of the term huh.. big data and could you explain to me what big data in briefly?

My Ayesh: Yah so, big data is something like large, massive datasets which cannot be dealt with traditional data management systems. These data sets can be structured, unstructured data and also can be semi-structured data. So, in our organization, big data mainly allows us to analyze large volumes of data, hmm, from various sources to gain valuable insights which can be used for, hmm. business intelligence and also in decision making.

Senash: Ok so the next question is which forms of data are considered as the most sensitive in your organization's big data environment?

Mr Ayesh: Basically, it is those financial data in the company and also the personal information of employees would be the most sensitive in the big data environment.

Senash: Alright so has your organization experienced any data breaches as a security challenge related to big data?

My Ayesh: No, in the last couple of months, we have not experienced any data breach in our organization. However, if there is any, it can cause a serious damage to the us and will expose some confidential details as well.

Senash: So do you think data breaches is a big data challenge for organizations?

Mr Ayesh: Yah of course, it can be a very significant challenge, I would say, especially organizations that deal with big data because all the sensitive information in the organization will be lost. So yah, I see it as a challenge

Senash: Ok so the next question is have your organization faced incidents of unauthorized access to big data?

Mr Ayesh: I mean, there have been few incidents like this, but they were not like serious situations but as you said earlier this can be significant big data challenge as well when comparing it with data breaches.

Senash: Ok sir so the next question is does your organization identify a lack of employee awareness as a challenge in big data security?

Mr. Ayesh: I don't think so. Hmm. It's because in our organization we provide training for employees regarding the organization practices as well provide awareness to big data security as well. So, we do provide awareness to all parts of security that can affect the company and not only regarding big data.

Senash: So, do you think that organizations identify a lack of employee awareness as a challenge in big data security?

Mr. Ayesh: Yah I mean it can be a big data challenge because if employees are not aware about these in detail, it can be very dangerous. So for example, hmmm, if employees are unaware of proper practices for managing sensitive data, such as huh.. encryption

technologies, they may unintentionally expose data to unauthorized access or mishandle it, resulting in big data security breaches. So yah, that could be another challenge.

This also reminds me of an incident which happened a few months ago where one of my friends and also a employee at the top hierarchy accidently exposed data in our organization. So yah, so there can be challenges like this.

Senash: Ohh Okay sir, so the next question is have your organization experienced compliance issues related to the security of big data?

Mr. Ayesh: Sorry?..

Senash: have your organization experienced compliance issues related to the security of big data?

Mr. Ayesh: No, not in LOLC because we are aware and follow the regulations very well. So, if hmm overall, if we talk about compliance issues, it can be a big data challenge because hmm, these regulations, which is in srilanka which is the The Personal Data Protection Act No. 9 of 2022 ('PDPA') of Sri Lanka provide strict requirements for protecting sensitive data, especially big data, and companies like us must follow them to avoid fines and any other legal consequences.

Senash: Ok sir so the next question is huh, hmm what other big data security challenges have been experienced by your organization? And can you please explain it briefly.

Mr. Ayesh: Yah so, if we talk about the data volume of big data it can be challenge, not just for us but also for any organization that handles big data. So, what I mean is that the amount of big data being generated is enormous, and maintaining, processing, and also analyzing such vast amounts requires tremendous computational resources and storage capacity right?, so yah so this can be challenge as well.

And also when talking about the data quality, ensuring data quality is a significant challenges in big data analytics as well. So what I mean is that With vast volumes of data arriving from diverse sources like different sources, there is a risk of mistakes and inconsistencies right?, so these poor data quality can lead to poor insights which can affect any organization

Senash:: So, is there any other security challenge? I mean you can give an answer in general, like in any organizations?

Yah sure, so data privacy can be challenge as well. I say this because, huh, as organizations collect and analyze massive volumes of data, maintaining data privacy and security becomes critical and very challenging, right? So there can be concerns with unauthorized access and data breaches as we talked about earlier, so yah.

Also, if we talk about the 3Vs of big data, the data variety can be a challenge as well. The speed at which the data is generated and needs to be processed can be another challenge. In my opinion, this is because real-time data processing and analytics may be needed for. Hmmm, specific applications that requires computing power and streaming analytics capabilities.

Senash: Ok sir,

Mr. Ayesh: Huh and also for organizations that use cloud-based big data platforms, ensuring cloud security can be a challenge as well.

Senash: so from the challenges you have mentioned, what do you think is the most significant security challenge with big data in your organization?

Mr. Ayesh: So in our organization, I would say that the challenge with data volume I talked earlier, would be the most significant challenge among those other challenges and if we talk about the most significant challenge in general, I think unauthorized access and maybe data compliance and data breaches would one of the most significant challenge. In general right?

Senash: Ok so the next question is has your organization implemented encryption as a mitigation strategy for big data security?

Mr. Ayesh: Of course, given the sensitivity of the data in our organization's big data environment, that includes all the financial data and all, huh this encryption measures such as SSL/TLS can help to protect the data from various threats, right? And also to ensure safety while transferring data as well.

So we use passwords, biometrics and all as well so yah.

Senash: What do you think about more advanced encryption techniques?

Mr. Ayesh: Yah so huh techniques like AES and all huh, homographic which basically use complex mathematical operations and symmetric encryptions can be the most innovative,

I would say techniques that can be used as well. And also data at transit can be secure during SSL and TLS while data at rest can then be secured using AES. Yah.

Senash: Alright. So hmm, the next question is does your organization implement data masking techniques to enhance big data security?

Mr. Ayesha: Yah, hmmm, huh, we somewhat use data masking technique something like generalization and all, so overall this method basically hides the data by modifying it and also while maintaining their usefulness, I would say, like for testing and analysis.

So, as I said earlier overall this technique will also help to mitigate those challenges that involves those security threats, huh... which is related to big data.

Senash: Ok so the next one is, does your organization conduct regular audits as a strategy to mitigate big data security challenges?

Mr. Ayesh:

Of course, obviously, we conduct regular audits, not just to identify and mitigate those big data related challenges, and also conduct huh... compliance audits, and in fact encryption audits as well so to ensure that we meet the regulations and standards, and also security assessments huh, hmmm to identify the vulnerabilities in big data environments, I would say.

Senash: Ok so the next question is does your organization provide employees with training on best practices for big data security?

Mr. Ayesh: Yes, we do Senash.

Senash: How does it work sir.

Mr. Ayesh: Yah so basically by providing training, the employees will be aware of these big data security challenges and in case a data breach or some other threat occurs, they will be ready for it right?, like they will know what to do and how to do it.

So we provide regular training programs and all including online initiatives and. It also reduces the human errors caused, like mistakes, like unauthorized access to big data. So yah providing training will be strategy to minimize the security challenge related to big data.

Senash: Ok sir so does your organization have a defined incident response plan for big data security incidents?

Mr. Ayesh: Yah we do, so like when a big data breach or any security incident occurs at LOLC, the incident response plan will allow us to manage this security incident effectively right?

Senash: Yah. And also what types of systems have you guys implemented

Mr. Ayesh: Hmm, we will be able to like make better decisions during a security incident, and mitigate these security challenges, risks before it becomes a major, I mean, a serious incident.

And also for your question, we have implemented IDS and IPS respectively and systems to identify any security incidents.

Mr. Ayesh:

Senash: Ok sir, so the next question is has your organization implemented access controls as a mitigation strategy for big data security?

Yes of course, we do. Not just in our organization and almost all of the organizations that handles big data have implemented access controls. I say this, because, access controls help to manage who has access to data, what actions they can do on it, and under what conditions. So, over here we implement RBAC, which is role-based access controls, yah so permissions are assigned to the employees huh depending on their roles and duties inside the organization. Roles such as data analyst, data scientist and so on have been defined and appropriate permissions have been assigned to each role. So yah.

And also we are using something called Multi-Factor Authentication, if you have heard of it. So, basically it will give access to a user at LOLC after he goes through few security measures.

Senash: Ok sir, so the final question is there any other mitigation strategies or measures implemented by your organization in order to mitigate the security challenges?

Mr. Ayesh: The other strategies used in LOLC where I work hmm, huh, hmmm,, huh the data quality assessment tools like I just can't remember the tool we use, but I would say we use these tools to find and fix any inconsistencies, mistakes, and inaccuracies in big data sets. We do, this to make sure their reliable and also to assure their reliability.

In general, I would think like there is something called a data governance framework, and huh..... anonymization techniques, huh, and advance analytics and machine learning algorithms can be implemented to identify any big data security threat and all.. So I think these, strategies and techniques will help to mitigate the big data security challenges.

And also when talking about these advanced techniques, this reminds me of the blockchain technology as well.

Senash: Ok sir so that's the ... huh or one more question sir

Mr. Ayesh: Yah.

Mr. Ayesh:

Senash: So from the following you have mentioned, what specific strategies or technologies has a significant impact in your organization to handle big data security challenges?

I would say in our organizations and also taking all the organizations that manages big data into account, access controls and encryption would be the most basic and also has a significant impact on organizations, when dealing with big data security challenges.

This is because, as I talked about earlier about encryption and access controls, these measures provide a greater security to manage these large volumes of data.

Senash: Ok sir, so that's the end of the questions

Mr. Ayesh: Ok Senash so

Senash: Yah sir, thank you for participating.

Mr. Ayesh Yah Senash, no problem. Thank you.

Senash: Thank you sir.



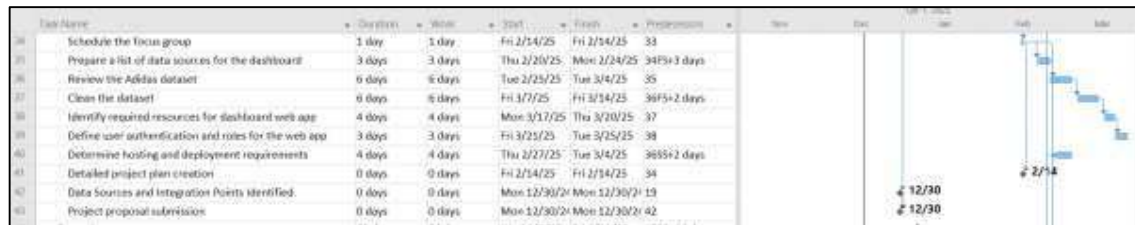
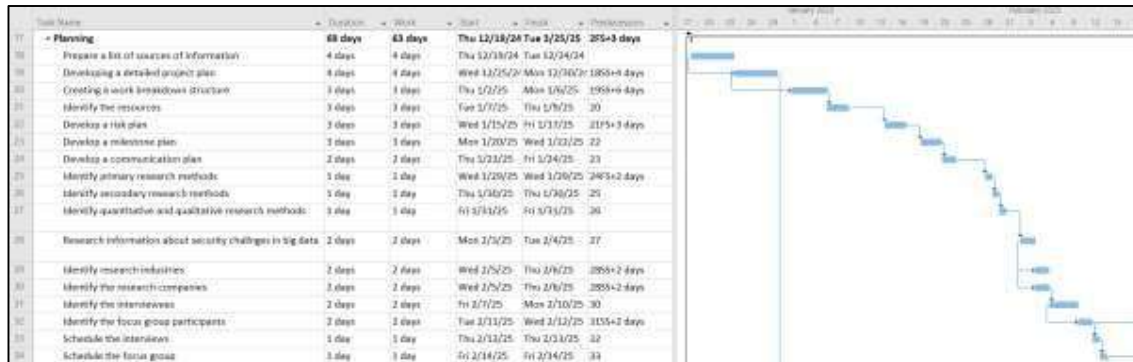
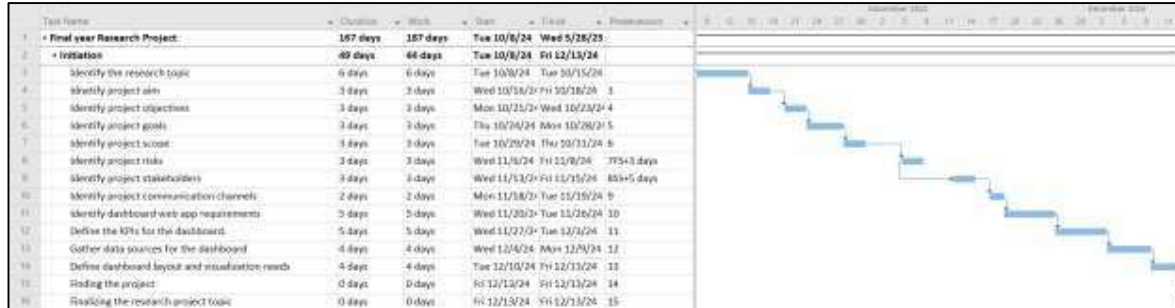
Mr. Ayesh:

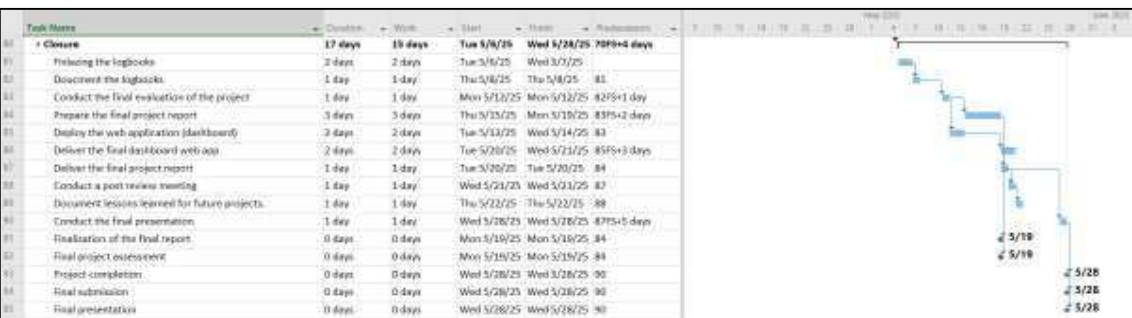
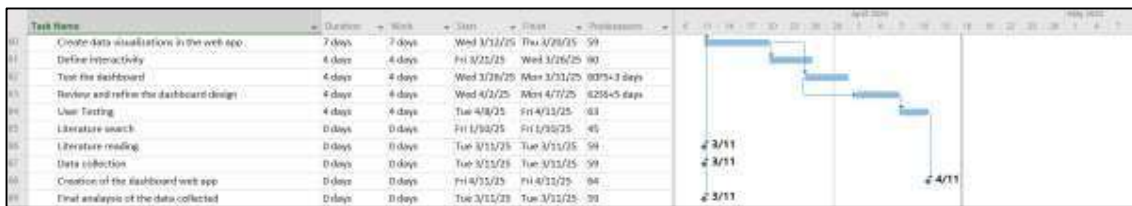
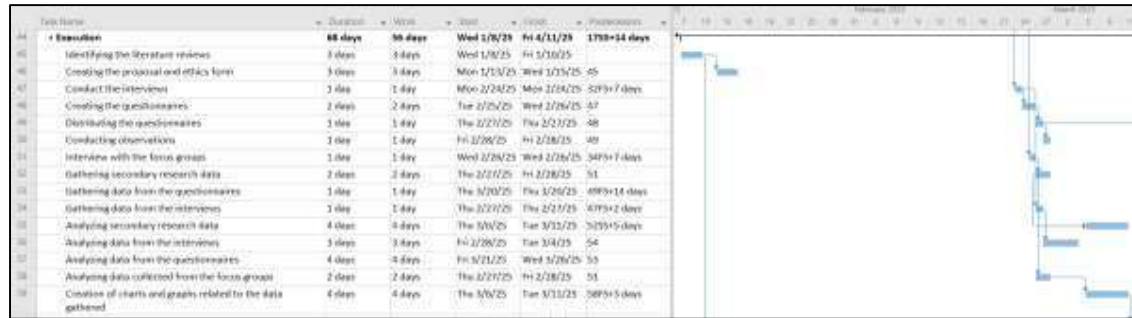
PROJECT PLAN



Gantt chart / Timeline

The project Gantt chart and the activities include:





The project tasks in a detailed format are as follows:

Task Mode	Task Name	Duration	Work	Start	Finish	Predecessors
1	Final year Research Project	167 days	187 days	Tue 10/8/24	Wed 5/28/25	
2	Initiation	49 days	44 days	Tue 10/8/24	Fri 12/13/24	
3	Identify the research topic	6 days	6 days	Tue 10/8/24	Tue 10/15/24	
4	Identify project aim	3 days	3 days	Wed 10/16/24	Fri 10/18/24	3
5	Identify project objectives	3 days	3 days	Mon 10/21/24	Wed 10/23/24	4
6	Identify project goals	3 days	3 days	Thu 10/24/24	Mon 10/28/24	5
7	Identify project scope	3 days	3 days	Tue 10/29/24	Thu 10/31/24	6
8	Identify project risks	3 days	3 days	Wed 11/6/24	Fri 11/8/24	7FS+3 days
9	Identify project stakeholders	3 days	3 days	Wed 11/13/24	Fri 11/15/24	8SS+5 days
10	Identify project communication channels	2 days	2 days	Mon 11/18/24	Tue 11/19/24	9
11	Identify dashboard web app requirements	5 days	5 days	Wed 11/20/24	Tue 11/26/24	10
12	Define the KPIs for the dashboard.	5 days	5 days	Wed 11/27/24	Tue 12/3/24	11
13	Gather data sources for the dashboard	4 days	4 days	Wed 12/4/24	Mon 12/9/24	12
14	Define dashboard layout and visualization needs	4 days	4 days	Tue 12/10/24	Fri 12/13/24	13
15	Finding the project	0 days	0 days	Fri 12/13/24	Fri 12/13/24	14
16	Finalizing the research project topic	0 days	0 days	Fri 12/13/24	Fri 12/13/24	15

Task Mode	Task Name	Duration	Work	Start	Finish	Predecessors
17	Planning	69 days	63 days	Thu 12/19/24	Tue 3/25/25	2FS+3 days
18	Prepare a list of sources of information	4 days	4 days	Thu 12/19/24	Tue 12/24/24	
19	Developing a detailed project plan	4 days	4 days	Wed 12/25/24	Mon 12/30/24	18SS+4 days
20	Creating a work breakdown structure	3 days	3 days	Thu 1/2/25	Mon 1/6/25	19SS+6 days
21	Identify the resources	3 days	3 days	Tue 1/7/25	Thu 1/9/25	20
22	Develop a risk plan	3 days	3 days	Wed 1/15/25	Fri 1/17/25	21FS+3 days
23	Develop a milestone plan	3 days	3 days	Mon 1/20/25	Wed 1/22/25	22
24	Develop a communication plan	2 days	2 days	Thu 1/23/25	Fri 1/24/25	23
25	Identify primary research methods	1 day	1 day	Wed 1/29/25	Wed 1/29/25	24FS+2 days
26	Identify secondary research methods	1 day	1 day	Thu 1/30/25	Thu 1/30/25	25
27	Identify quantitative and qualitative research methods	1 day	1 day	Fri 1/31/25	Fri 1/31/25	26
28	Research information about security challenges in big data	2 days	2 days	Mon 2/3/25	Tue 2/4/25	27
29	Identify research industries	2 days	2 days	Wed 2/5/25	Thu 2/6/25	28SS+2 days
30	Identify the research companies	2 days	2 days	Wed 2/5/25	Thu 2/6/25	28SS+2 days
31	Identify the interviewees	2 days	2 days	Fri 2/7/25	Mon 2/10/25	30
32	Identify the focus group participants	2 days	2 days	Tue 2/11/25	Wed 2/12/25	31SS+2 days
33	Schedule the interviews	1 day	1 day	Thu 2/13/25	Thu 2/13/25	32
34	Schedule the focus group	1 day	1 day	Fri 2/14/25	Fri 2/14/25	33

Task Mode	Task Name	Duration	Work	Start	Finish	Predecessors
35	Prepare a list of data sources for the dashboard	3 days	3 days	Thu 2/20/25	Mon 2/24/25	34FS+3 days
36	Review the Adidas dataset	6 days	6 days	Tue 2/25/25	Tue 3/4/25	35
37	Clean the dataset	6 days	6 days	Fri 3/7/25	Fri 3/14/25	36FS+2 days
38	Identify required resources for dashboard web app	4 days	4 days	Mon 3/17/25	Thu 3/20/25	37
39	Define user authentication and roles for the web app	3 days	3 days	Fri 3/21/25	Tue 3/25/25	38
40	Determine hosting and deployment requirements	4 days	4 days	Thu 2/27/25	Tue 3/4/25	36FS+2 days
41	Detailed project plan creation	0 days	0 days	Fri 2/14/25	Fri 2/14/25	34
42	Data Sources and Integration Points Identified	0 days	0 days	Mon 12/30/24	Mon 12/30/24	19
43	Project proposal submission	0 days	0 days	Mon 12/30/24	Mon 12/30/24	42
44	Execution	68 days	56 days	Wed 1/8/25	Fri 4/11/25	17SS+14 days
45	Identifying the literature reviews	3 days	3 days	Wed 1/8/25	Fri 1/10/25	
46	Creating the proposal and ethics form	3 days	3 days	Mon 1/13/25	Wed 1/15/25	45
47	Conduct the interviews	1 day	1 day	Mon 2/24/25	Mon 2/24/25	32FS+7 days
48	Creating the questionnaires	2 days	2 days	Tue 2/25/25	Wed 2/26/25	47
49	Distributing the questionnaires	1 day	1 day	Thu 2/27/25	Thu 2/27/25	48
50	Conducting observations	1 day	1 day	Fri 2/28/25	Fri 2/28/25	49
51	Interview with the focus groups	1 day	1 day	Wed 2/26/25	Wed 2/26/25	34FS+7 days
52	Gathering secondary research data	2 days	2 days	Thu 2/27/25	Fri 2/28/25	51
53	Gathering data from the questionnaires	1 day	1 day	Thu 3/20/25	Thu 3/20/25	49FS+14 days

Task Mode	Task Name	Duration	Work	Start	Finish	Predecessors
54	Gathering data from the interviews	1 day	1 day	Thu 2/27/25	Thu 2/27/25	47FS+2 days
55	Analyzing secondary research data	4 days	4 days	Thu 3/6/25	Tue 3/11/25	52SS+5 days
56	Analyzing data from the interviews	3 days	3 days	Fri 2/28/25	Tue 3/4/25	54
57	Analyzing data from the questionnaires	4 days	4 days	Fri 3/21/25	Wed 3/26/25	53
58	Analyzing data collected from the focus groups	2 days	2 days	Thu 2/27/25	Fri 2/28/25	51
59	Creation of charts and graphs related to the data gathered	4 days	4 days	Thu 3/6/25	Tue 3/11/25	58FS+3 days
60	Create data visualizations in the web app	7 days	7 days	Wed 3/12/25	Thu 3/20/25	59
61	Define interactivity	4 days	4 days	Fri 3/21/25	Wed 3/26/25	60
62	Test the dashboard	4 days	4 days	Wed 3/26/25	Mon 3/31/25	60FS+3 days
63	Review and refine the dashboard design	4 days	4 days	Wed 4/2/25	Mon 4/7/25	62SS+5 days
64	User Testing	4 days	4 days	Tue 4/8/25	Fri 4/11/25	63
65	Literature search	0 days	0 days	Fri 1/10/25	Fri 1/10/25	45
66	Literature reading	0 days	0 days	Tue 3/11/25	Tue 3/11/25	59
67	Data collection	0 days	0 days	Tue 3/11/25	Tue 3/11/25	59
68	Creation of the dashboard web app	0 days	0 days	Fri 4/11/25	Fri 4/11/25	64
69	Final analysis of the data collected	0 days	0 days	Tue 3/11/25	Tue 3/11/25	59

Task Mode	Task Name	Duration	Work	Start	Finish	Predecessors
70	Monitoring and controlling	9 days	9 days	Thu 4/17/25	Tue 4/29/25	44FS+3 days
71	Confirm the interviewees	1 day	1 day	Thu 4/17/25	Thu 4/17/25	
72	Time management	1 day	1 day	Fri 4/18/25	Fri 4/18/25	71
73	Issue management	1 day	1 day	Mon 4/21/25	Mon 4/21/25	72
74	Quality management	1 day	1 day	Tue 4/22/25	Tue 4/22/25	73
75	Continuously update the stakeholder on project progress	1 day	1 day	Wed 4/23/25	Wed 4/23/25	74
76	Review and analyze the project performance	1 day	1 day	Thu 4/24/25	Thu 4/24/25	75
77	Progress review meetings conducted	1 day	1 day	Fri 4/25/25	Fri 4/25/25	76
78	Documentation management including logbooks	1 day	1 day	Mon 4/28/25	Mon 4/28/25	77
79	Address any deviations from the project plan	1 day	1 day	Tue 4/29/25	Tue 4/29/25	78

80	📅	* Closure	17 days	15 days	Tue 5/6/25	Wed 5/28/25	70FS+4 days
81	📅	Finalizing the logbooks	2 days	2 days	Tue 5/6/25	Wed 5/7/25	
82	📅	Document the logbooks	1 day	1 day	Thu 5/8/25	Thu 5/8/25	81
83	📅	Conduct the final evaluation of the project	1 day	1 day	Mon 5/12/25	Mon 5/12/25	82FS+1 day
84	📅	Prepare the final project report	3 days	3 days	Thu 5/15/25	Mon 5/19/25	83FS+2 days
85	📅	Deploy the web application (dashboard)	2 days	2 days	Tue 5/13/25	Wed 5/14/25	83
86	📅	Deliver the final dashboard web app	2 days	2 days	Tue 5/20/25	Wed 5/21/25	85FS+3 days
87	📅	Deliver the final project report	1 day	1 day	Tue 5/20/25	Tue 5/20/25	84
88	📅	Conduct a post review meeting	1 day	1 day	Wed 5/21/25	Wed 5/21/25	87
89	📅	Document lessons learned for future projects.	1 day	1 day	Thu 5/22/25	Thu 5/22/25	88
90	📅	Conduct the final presentation	1 day	1 day	Wed 5/28/25	Wed 5/28/25	87FS+5 days
91	📅	Finalization of the final report	0 days	0 days	Mon 5/19/25	Mon 5/19/25	84
92	📅	Final project assessment	0 days	0 days	Mon 5/19/25	Mon 5/19/25	84
93	📅	Project completion	0 days	0 days	Wed 5/28/25	Wed 5/28/25	90
94	📅	Final submission	0 days	0 days	Wed 5/28/25	Wed 5/28/25	90
95	📅	Final presentation	0 days	0 days	Wed 5/28/25	Wed 5/28/25	90

BUDGET PLAN

1. Personnel Costs

Item	Description	Cost (USD)
Student Researcher	6 months of full-time work (stipend)	\$0
Supervisor Consultation	Honorarium for guidance and reviews	\$0 (University supervisor)
Subtotal		\$0

Software & Tools

Item	Description	Cost (USD)
Python Libraries (Pandas, Plotly, etc.)	Open-source (no cost)	\$0
Streamlit & Flask Hosting	Free tier (Streamlit Sharing/Heroku)	\$0
MongoDB Atlas (Database)	Free tier (512MB storage)	\$30
Subtotal		\$30

Hardware & Infrastructure

Item	Description	Cost (USD)
Laptop/PC for Development	Already available (no cost)	\$0
Cloud Storage (Backups)	Google Drive (15GB free)	\$0
Subtotal		\$0

Data Acquisition

Item	Description	Cost (USD)
Adidas Sales Dataset	Downloaded from Kaggle (free)	\$0
Survey Platform (Google Forms)	Free tier	\$0
Subtotal		\$0

Travel & Communication

Item	Description	Cost (£)
Internet Costs	Existing subscription	£0
Interview Transportation	Local travel for stakeholder meetings	\$5
Sharing surveys	Transportation to several organizations to request to fill the survey	\$10

Miscellaneous

Item	Description	Cost (USD)
Printing & Documentation	Thesis binding, posters, etc.	\$10
Contingency Fund	Unforeseen expenses	\$50
Subtotal		\$60

Total Estimated Budget: \$105

Justification of Costs:

- Personnel: Majority allocated to the researcher's stipend for 6 months of dedicated work.
- Software/Hardware: Open-source tools and free-tier cloud services minimized expenses.
- Data: Publicly available datasets (Kaggle) and free survey tools were used.
- Contingency: Reserved for unexpected costs (e.g., additional cloud storage, depolyment).

Cost-Saving Measures:

- Utilized free-tier hosting (Streamlit, MongoDB) and open-source libraries.
- Leveraged existing hardware (personal laptop) and internet.
- Conducted the interview at a fixed place to reduce travel costs.



Monitoring the Progress of Final Year Projects ***Supervisor Log Sheet***

BSc. (Hons) IT – University of West London, UK

Project Title:

Big Data Challenges and Mitigation Strategies in organizations: A Data
Driven Analysis with Industry Insights and development of
Sales Dashboard Web App using big data.

Name of the Student:

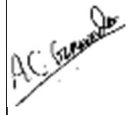
Punchawarige Mahisa Senash

Name of the Supervisor:

Mr. Ashen Chathuranga

**To be filled and signed by the supervisor*

Minimum 2 meetings per month

Meeting	Criteria	Suggestions	Actions	Date	Signature
<i>Proposal Stage</i>					
1.	Project topic approval	The topic is in line with the goals of the program. Make sure you concentrate on both the practical (dashboard creation) and theoretical (Big Data problems) elements. Suggested to reduce the scope of the project. Overall, approved the topic.	Refined scope to balance the research and the implementation	25/03/2025	
2.	Project proposal approval	Strong proposal. The proposal was clearly structured with defined aims, objectives, methodology, and tools. The supervisor suggested including more academic references and elaborating on the tools and technologies to be used for dashboard development.	Additional academic sources were included in the literature review. Technologies such as Python, Excel, and Power BI were finalized for implementation.	30/03/2025	

<i>Interim & Final Stage</i>					
3.	Literature review (Gathered resource documents)	Ensure diversity of sources – include industry reports, whitepapers, and peer-reviewed journals.	Collected and reviewed academic and industry resources on big data security challenges and mitigation strategies along with the a literature review for the development of the dashboard web app	03/04/2025	
4.	Literature review (Report)	Supervisor emphasized the need for critical analysis and synthesis of the reviewed literature. Gaps in current research should be clearly identified and aligned with the project focus.	Drafted the literature review section with organized subsections: Big Data characteristics, security challenges, mitigation techniques, and visualization strategies. Gaps related to practical implementation in small-to-medium enterprises highlighted.	07/04/2025	
5.	Approach (Users, Input, Output, Technologies Used)	Recommended clearly defining user personas, system inputs (sales data), and expected outputs (dashboards, predictions). Justify technology choices.	Documented system inputs (CSV sales data), defined users (analysts, business users), and outputs (dashboards, predictions). Chose Streamlit for UI, Pandas for data processing, Prophet and Random Forest for ML, and MongoDB for secure login storage. Flask + HTML used to build a custom login page.	10/04/2025	

6.	Project Design	Advised to include architecture diagrams, system flowcharts, and wireframes aligned with project goals and technologies used	Wireframes that matched the technologies and project objectives. UML diagrams were used to design and document the architecture (use case, data flow).	15/04/2025	
			Created user interface draft for the login and dashboard of Streamlit. Flask and MongoDB were used to illustrate the backend structure.		
7.	Implementation (Algorithms, Flow charts, System, etc.)	Supervisor advised testing ML models and updating flowcharts to reflect the flow of user login, data processing, and visualization	Pandas was used to implement data transformation and preprocessing. created Random Forest for predictive analysis and Prophet-based time-series forecasting. Streamlit dashboard pipeline and ML integration are depicted.	25/04/2025	
8.	Implementation (System Development)	Ensure system modules (login, dashboard, predictions) are integrated and tested. Debug UI and confirm MongoDB authentication works as intended.	Integrated Streamlit dashboard with backend Flask login module. Connected MongoDB for secure user login. Verified ML model outputs are rendered dynamically on the dashboard.	01/05/2025	

9.	Implementation (Finalization)	Record unit and integration tests, such as those for forecast accuracy, data input handling, and login validation. Add screenshots.	Improved Streamlit responsiveness and enhanced chart style with Seaborn/Matplotlib and Plotly. Improved error handling for incorrect inputs and an improved login experience.	10/05/2025	
10.	Evaluation (Testing evidence)	The supervisor requested that all test cases (both unit and integration) be documented. Stress unforeseen user inputs and edge scenarios.	Compiled comprehensive test logs, including screenshots. (More than 50 Test cases) Evaluated the performance and the reliability of the predictive models (MAE, RSME) and the test cases of the dashboard web app. Critical bugs were identified and resolved. Verification steps recorded	14/05/2025	

11.	Conclusion (Result interpretation, Achievement of objectives, Limitations) & Further work	Summarize achievements against objectives. Highlight limitations (e.g., real-time data not used) and propose future enhancements such as Real-time Analytics Integration	Advanced big data mitigation strategies were identified including encryption techniques like AES, tokenization, advanced ML/AI models to mitigate big data challenges. Further, the report conclusion finalized. Included a comparison of objectives vs outcomes, noted limitations, and proposed enhancements like live data and advanced ML models like XGBoost, LSTM neural networks, or ensemble methods to potentially improve forecasting accuracy	26/05/2025	
12.	Guidance for the final viva	Supervisor recommended practicing the presentation thoroughly and preparing responses to common academic questions, especially regarding methodology and results.	Prepared PowerPoint slides, rehearsed delivery, and conducted a mock Q&A session. Prepared demo showcasing the Streamlit user interface, Flask/MongoDB login mechanism, and sales forecasts. ready to	28/05/2025	
			confidently respond to queries that are conceptual and technical.		